

CSCI441 Software Engineering

“WasteLesss”

Group: Team C

Pavel Martinez

Brandon Tilley

Larisa Smith

Janelle Holcomb

May 4, 2025

https://github.com/Larisa24fhsu/GroupC_CSCI441_SoftwareEngineeringProject.git

t

Breakdown of Individual Contributions:

All team members contributed equally throughout the course of this project. Please see the Responsibility Matrix below for a more detailed breakdown of the tasks required in the report.

Responsibility Matrix:

Key	
R	Responsible: The person or people who complete the task
A	Accountable: The person who has the final say on the task
C	Consulted: The person who provides advice or expertise on the task
I	Informed: The person who is kept up to date on the task's progress

You can see the full matrix using the following link [Raci Matrix.xlsx](#)

RACI Matrix					
Item	Larisa Smith	Pavel Martinez	Brandon Tilley	Janelle Holcomb	
Report 1 Part 1					
Customer Problem Statement of Requirements					
Problem Statement	I	C	A/R	C	
Decomposition Explanation	I	A/R	C	I	
Glossary of Terms	A/R	A/R	A/R	A/R	
Goals, Requirements, and Analysis					
Business Goals	C	I	C	A/R	
Enumerated Functional Requirements	A/R	I	I	I	
Enumerated Non-Functional Requirements	C	A/R	I	I	
User Interface Requirements	I	I	A/R	C	
Goals, Requirements, and Analysis					
Stakeholder Identification	A/R	A/R	A/R	A/R	
Actors and Goals	C	A/R	C	C	
Use Cases - Description	C	I	A/R	C	
Use Case Diagrams	I	I	C	A/R	
Traceability Matrix	I	A/R	C	I	
Fully-Dressed Description	A/R	C	I	I	
System Sequence Diagrams	A/R	C	I	I	
User Interface Specification	I	I	A/R	C	
Preliminary Design	C	I	A/R	A/R	
User Effort Estimation	A/R	I	C	I	
System Architecture and System Design					
Identifying Subsystems	C	I	C	A/R	
Architecture Styles	C	C	A/R	I	
Mapping Subsystem to Hardware	A/R	C	I	C	
Connectors and Network Protocols	A/R	A/R	A/R	A/R	
Global Control Flow	I	A/R	C	C	
Hardware Requirements	A/R	C	C	I	
Analysis and Domain Modeling					
Conceptual Model	I	C	A/R	I	
System Operation Contracts	A/R	I	I	C	
Data Model and Persistent Data Storage	I	A/R	C	I	
Mathematical Model	C	C	C	A/R	
Algorithms, Data Structures, User Interface Design & Implementation, Test Designs					
Algorithms	I	I	I	A/R	
Data Structures	I	I	I	A/R	
Concurrency	A/R	I	I	I	
User Interface Design and Implementation	I	I	A/R	I	
Test Designs	A/R	A/R	I	I	
Project Management					
Teamwork Delegation	A/R	A/R	A/R	A/R	
Meeting Overview	A/R	I	I	I	
Proof-reading and Editing	A/R	I	I	C	
Project Plan	A/R	I	I	I	

Table of Contents

Summary of Changes	4
Customer Problem Statement of Requirements	5
Problem Statement	5
Decomposition into Sub-Problems	8
Glossary of Terms	8
Goals, Requirements, and Analysis	11
Business Goals	11
Enumerated Functional Requirements	11
Enumerated Non-Functional Requirements	12
User Interface Requirements	13
Functional Requirement Specification and Use Cases	15
Stakeholders	15
Actors and Goals	16
Use Cases	17
Casual Description	17
Use Case Diagram	18
System Components	19
Traceability Matrix	20
Fully Dressed Descriptions	20
System Sequence Diagrams	26
User Interface Specification	29
Preliminary Design	31
User Effort Estimation	35
System Architecture and System Design	37
Identifying Subsystems	37
Architecture Styles	38
Connectors and Network Protocols	39
Global Control Flow	40
Execution orderliness:	40
Time dependency:	40
Hardware Requirements	40
Analysis and Domain Modeling	41
Conceptual Model	41
Concept Definitions	42
Association Definitions	42
Attribute Definitions	43
Traceability Matrix	44

System Operations Contracts	44
Data Model and Persistent Data Storage	47
Mathematical Model	49
Basic Mathematical Model for Inventory Optimization	49
Interaction Diagrams	51
Class Diagram and Interface Specification	56
Class Diagram	56
Data Types and Operation Signatures	56
Traceability Matrix	59
Design Patterns	61
Object Constraint Language (OCL) Contracts	62
Algorithms and Data Structures	63
Algorithms	63
Data Structures	68
Why We Chose These Data Structures	70
Concurrency: Does your system use multiple threads?	70
User Interface Design and Implementation	71
Test Designs	77
Testing Coverage	80
Integration Testing	80
Outline of Algorithm and System Specific Tests	81
Project Management	82
Merging the Contributions from Individual Members	82
History of Work, Current Status, and Future Work	82
History of Work	82
Future Work	84
References	86

Summary of Changes

As for updates, we all updated our originally defined topics as shown in the above RACI Matrix

User Effort Estimation (p. 35)

- Revised to align with the system's actual functionality and recent changes.

Connectors and Network Protocols (p. 39)

- Updated to reflect actual implementation:
 - Replaced Java and JDBC with Node.js, PostgreSQL (pg), and REST APIs over HTTP/HTTPS
- Clarified connection flows between frontend, backend, and database

Conceptual Model (p. 41)

- Expanded to reflect register/login functionality
- Updated to align with the evolved controller structure

Concept Definitions / Association / Attribute Definitions (p. 42–43)

- Revised to support new conceptual model elements
- Captures domain-specific additions (e.g., alert types, user credentials)

Traceability Matrix (p. 44, 59)

- Added links between new domain concepts and corresponding software classes
- Updated entries for alert controller and user authentication

Mathematical Model (p. 49)

- Included the actual EOQ (Economic Order Quantity) formula used in the alert system.
- Explained how the EOQ formula is integrated into backend logic.
- Provided an example of how EOQ results are presented to users.
- Noted a planned future feature: stock availability check before order placement.

Interaction Diagrams (p. 51)

- Updated to include design pattern annotations where relevant.
- Applied minor modifications to reflect the implemented system behavior.

Class Diagrams (p. 56)

- Added updates to the Register control.
- Diagram now reflects:

- User registration options (username and password)
- System access permissions

Design Patterns (p. 61)

- Documented implemented design patterns throughout the system

Object Constraint Language Contracts (p. 62)

- Added OCL (Object Constraint Language) contracts for key interactions

Test Designs (p. 77)

- Updated:
 - TC-4: Adjusted to align with the current API retrieval verbiage.
 - TC-5: Updated to reflect the alert method currently implemented.

Customer Problem Statement of Requirements

Problem Statement

Poor inventory management can result in profit loss for companies. These profit losses can be in the form of inventory carrying costs or inventory write-offs. Profit losses can vary by industry and business size, however it can comprise of 20% to 30% of the total inventory value and increase the longer an item is stored. The current industry has different methods of tracking inventory like barcode scanners, RFID Tags, QR codes, or even manual tracking. With many different business sizes, the current methods of tracking inventory do not address the specific needs of these businesses.

Businesses are struggling to manage inventory effectively, leading to increased costs, wasted products, and missing sales opportunities. Many companies face challenges like overstocking, running out of popular items, and dealing with perishable goods that expire before they can be sold. Without a real-time system to track inventory movement and provide alerts, businesses are forced to rely on *outdated* or *manual* methods, which can lead to mistakes and financial loss.

What is inventory Management?

Inventory management is the practice of overseeing and controlling the movement of goods and materials within an organization. It includes tracking stock levels, managing product ordering and storage, and ensuring the right quantities are available at the right time to meet customer demand. The primary goal of inventory management is to enhance operational efficiency, reduce costs, and prevent issues like overstocking or stockouts. Effective inventory

management enables businesses to strike a balance between having sufficient products to meet demand and minimizing the costs of holding excess stock.

Grocery Store Manager:

As a grocery store manager, the difficulties I face are managing perishable inventory, ensuring proper stock levels, and pushing old products. It is always a challenge when dealing with perishable goods because our customers expect a certain quality out of our products. Sometimes if a customer sees a product has abnormalities, they are less likely to buy it. This causes the item to spoil and be thrown away. I would like an easy way to deal with products that are close to expiration date but can still profit the company even if it is not monetary. With popular items that sell frequently we sometimes discourage customers who cannot find the item and our employees are not able to communicate with the customer when we get more inventory to satisfy the customer. Sometimes during the holiday season, we buy seasonal items, but it can be tricky because we don't know if the item will be popular. Sometimes we are left over with excessive seasonal inventory, and we do not know what to do with it, since we do not have the capacity to store the product for a whole year. These are problems as a grocery store manager that I would like to solve.

Grocery store employees:

As a grocery store employee, I face dissatisfied customers because of lack of inventory information. There are times when a customer will see a product online, then come to the store to buy it but they cannot find it on the shelf. When they ask me for help, I check the locations it could be and then check the back to see if it has been misplaced. When I cannot find the specific product, the customer gets incredibly upset for wasting their time. When I am walking through the store sometimes, I will notice a product that has been on the shelf for several weeks, long enough for it to collect dust. Other times I find myself opening duplicate orders of products because of miscommunication and now we are stuck with extra products that we cannot put on the shelves because there is not enough space.

Restaurant Owner:

As a restaurant owner, keeping track of my inventory is very hard. Unlike a grocery store, I don't just sell items, I use ingredients to make meals, and if I run out of something, I can't serve certain dishes. There have been times when I ran out of key ingredients during a dinner rush, and I had no way of knowing ahead of time. On the flip side, I also have food that spoils before I can use it, and that's just money wasted. I need a system that helps me track what's being used and what's about to expire. It would also be helpful if it could remind me when to reorder things, so I don't run out unexpectedly. My staff is too busy to manually check everything all the time, so I need something that makes the process easier and helps me cut down on waste.

Warehouse Manager:

As a warehouse manager, I make sure all the products are where they need to be and get shipped out on time. The problem is that without a good system, things get lost, and we don't always know what's close to expiring until it's too late. Sometimes we have too much of one thing and not enough of another, and that slows everything down. When we accidentally send out something that's about to expire, we end up with upset customers and wasted money. It would help if we had a way to track what is going in and out in real-time and get alerts before things expire. That way, we could rotate stock better and make sure nothing goes to waste.

Bank Manager:

As a bank manager, I need to make sure we have enough office supplies, including paper, pens, envelopes, and printer ink, to keep operations running smoothly. Running out of essential supplies slows down daily tasks and frustrates employees. Right now, we track inventory manually, and sometimes we don't realize we're low on something until it's already gone.

Another challenge is budgeting for office supplies. Without a clear system, we might over-order some items while running out of others. If we had a system that could track supply levels, send alerts when stock is low, and help with ordering the right amounts, it would save time and reduce waste.

In addition to office supplies, we also provide items for customer service, like coffee, coffee filters, cups, and stirrers. We offer suckers, mints, and other small treats for customers, but sometimes we run out unexpectedly, which can be disappointing for regular visitors. It would be helpful to have a way to track these items alongside our office supplies so we can reorder them before they run out. A system that keeps everything organized in one place and automates reordering would make supply management much easier!

Developed Problem Statement:

Customers are experiencing difficulty making decisions on how to resolve inventory carrying cost, inventory obsolescence, and perishable inventory. Other customers are experiencing difficulties in tracking the amount of time inventory has been put away, that includes perishable and non-perishables. Customers with unique needs have difficulty with inventory that has specific time interval values since they cannot set a unique time interval to alert them of aged or lost inventory, leading to a breakdown in communicating accurate information to the correct department. This is leading to customers having significant waste from overstocking, expired products, poor demand forecasting, extensive carry overflow and loss of product. This is resulting in loss of profits and missed opportunities within the business.

To improve inventory efficiency, reduce waste, reduce profit loss and improve decision making the system will provide real time tracking and notification with alternative methods of inventory handling. To improve customer specific needs the system will set item profiles to record specific information for their industry. By offering better insights and alternative inventory management solutions, businesses can make informed decisions, reduce unnecessary

costs, and keep both customers and employees satisfied. This would allow employees to be able to locate items faster, while customers would have a smoother shopping experience. With smarter inventory management, businesses can reduce costs, improve efficiency, and create a more reliable system for both the employees and the customers.

Decomposition into Sub-Problems

Customers want to be able to:

- Set item profiles to:
 - o Identify whether items are perishable or non-perishable.
 - o Set unique time interval values for shelf-life of perishable items.
 - o Set unique time interval alerts for aged and lost inventory.
 - o Set margins to decide how much inventory needs to be ordered.
 - o Set how much space is dedicated for unique items.
 - o Decide what departments should receive specific alerts.
- Time stamp inventory when it was received to:
 - o Track the amount of time inventory has been put-away.
 - o Track the shelf-life of perishable inventory.
 - o Track the variance in time between received to put-away, or shipped, to identify lost inventory.
- Search for inventory currently available in-stock.
- View in-transit inventory with estimated time of arrival.

Glossary of Terms

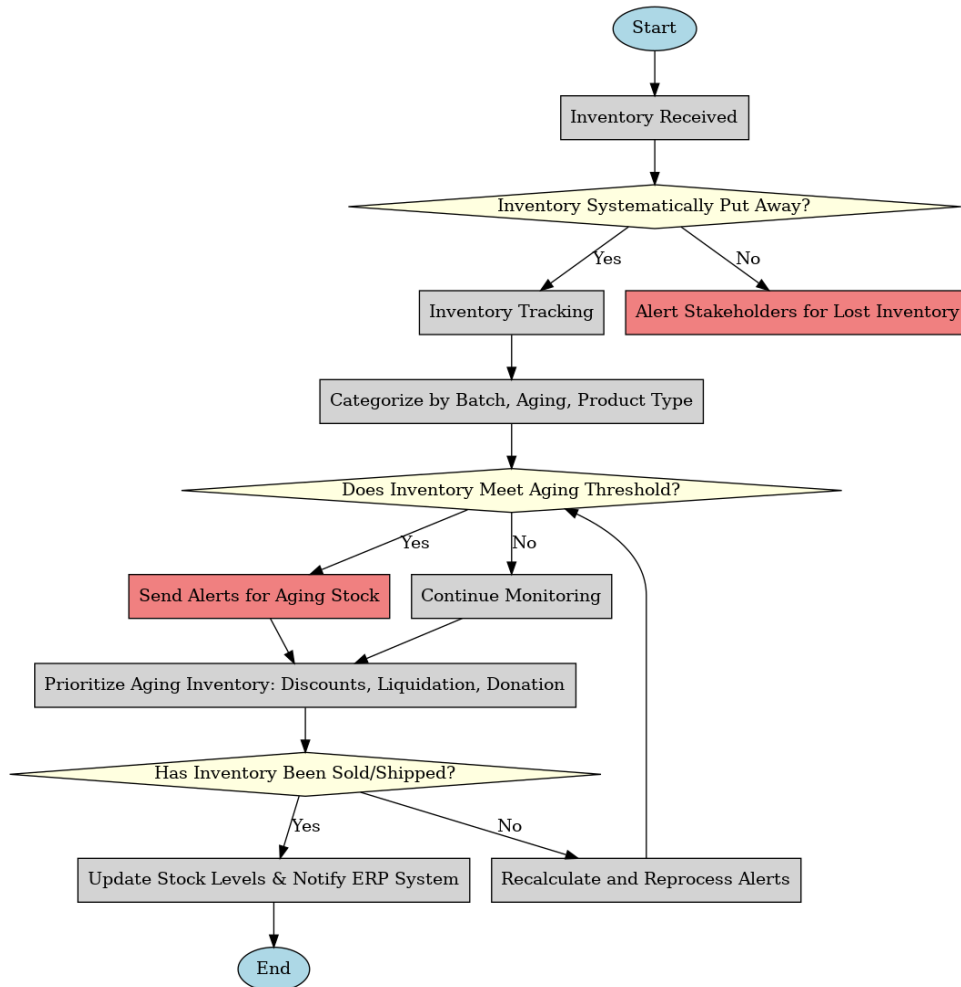
Term	Definition
Active module	The physical space where broken down cases of items are held.
Aged inventory	Inventory received and put-away but has not shipped for a measurable amount of time.
Allocated	A status that creates a task for items that are put-away to distribute those items to customers.
API - Application Programming Interface	A type of software that allows different computer programs or computers to communicate with each other.
Application Layer	Essentially acting as the bridge between the user and the underlying network infrastructure.
Automated Notifications	Alerts generated by the system to inform users when inventory levels are low, items are about to expire, or restocking is needed.
Business divisions	A distinct part of a company that may operate as its own legal entity or under the same name
Carrying Costs	The expenses of storing and maintaining inventory, including warehousing, insurance, and depreciation.
Central Processing Unit (CPU)	Primary component of a computer that acts as its “control center.”

Term	Definition
Client Modules	A software component that allows a client to interact with a server.
Client-side Component	Allows you to create an interactive user interface that is pre-rendered on the server and can use client-side JavaScript to execute in the browser
Current Stock	Inventory that is currently available to use or ship
Customer	A person or organization that buys goods.
Customer Profile Portal	A secure online space where customers can access their account information and interact with a company.
Customer Service Supplies	Non-essential but customer-facing inventory like coffee, cups, mints, and other items that enhance the customer experience.
Data Access Layer	A layer of a computer program which provides simplified access to data stored in persistent storage of some kind, such as an entity-relational database.
Distribution center	A place where a business receives, stores, and ships inventory.
Event Driven System	A software design that uses events to trigger actions between applications and devices.
Floor space	The physical space where distribution center operations are conducted.
Get Request	An HTTP method used to fetch information from an API.
HTTPS - Hypertext Transfer Protocol Secure	An extension of the Hypertext Transfer Protocol. It uses encryption for secure communication over a computer network, and is widely used on the Internet.
Inventory	A list of items that are sold to customers.
Inventory Management	The process of tracking, ordering, and maintaining stock levels to ensure necessary supplies are available while minimizing waste and overstock.
Inventory Obsolescence	The state of stock is becoming outdated, unsellable, or no longer needed, leading to financial losses if not managed properly.
Inventory Write-offs	The loss of inventory due to damage, expiration, or obsolescence, requiring it to be removed from records.
JDBC - Java Database Connectivity	An application programming interface for the Java programming language which defines how a client may access a database.
JSON - JavaScript Object Notation	Text-based format for storing and exchanging data.
Lost inventory	Items that were received but did not ship and were not put-away in a dedicated physical location.
Manual Tracking	Traditional, non-automated methods of inventory management, often involving spreadsheets, handwritten logs, or physical checks.
Modem	A networking device that is used to connect devices connected in the network to the internet.
Non-perishable inventory	Inventory that does not expire after any amount of time.
Orders	Requests made by customers for specific inventory.
Overstock	The act of adding items to inventory that are already available in abundance.
Perishable inventory	Inventory that becomes no longer sellable after a certain amount of time.

Term	Definition
Pixel	A minute area of illumination on a display screen, one of many from which an image is composed.
Presentation Layer	Topmost layer of an application, responsible for how data is displayed to the user through the user interface (UI), including elements like graphical design, input fields, and visual formatting.
Put-away	Items that have been received and are now physically in a dedicated location either in reserve racking or the active module.
Random Access Memory (RAM)	Short-term memory bank in a computer.
Real-Time Tracking	A system feature that provides live updates on inventory levels, movement, and availability to improve decision-making.
Received	Items transitioning from in-transit to the distribution center to arrived at distribution center.
Reserve racking	The physical space where full cases of items are held in inventory.
REST API	an application programming interface (API) that uses the Representational State Transfer (REST) architectural style.
Retail units	The items that make up the inventory
Router	A networking device that forwards data packets between computer networks.
Server Side Components	Components that run on a server, rather than on the client's browser.
Shipped	Sending items to customers.
SQL - Structured Query Language	Domain-specific language used to manage data, especially in a relational database management system.
Stock Rotation	A practice of organizing inventory so that older products are used or sold first to prevent expiration or obsolescence.
Sub-packages	Any package directly nested within another package.
Subsystem	A self contained system within a larger system.
Supply Budgeting	The process of planning and managing expenses related to purchasing necessary supplies while avoiding overspending.
System-wide Setting	Configuration option that applies to the entire system, affecting all users and components within it, rather than just a specific user or application
UML Package Diagram	Structural diagram in the Unified Modeling Language (UML) that visually represents the organization of a system by grouping related elements like classes, interfaces, and other packages into logical units called "packages,"
Warehouse	Synonym for distribution center.
Waste	Planning carelessly or with no purpose.
Web Application	Software program that runs in a web browser and allows users to perform tasks online.

Goals, Requirements, and Analysis

Business Goals



Enumerated Functional Requirements

Identifier	Priority	Requirement	Test Case
REQ1	3	Enable customers to add product specifications tailored to their needs. Allow for importation of data files for importing inventory	User input to assign goods by category and cost Inventory data sheet import for customizable options
REQ2	4	Customizable aging criteria for the customer to all for properly identify different product types and age criteria based on their value to the customer to optimize total stock ABC Analysis (Jenkins, 2024)	System to allow user to add products into specific grouping based on ABC analysis - "A" Products, "B" Products and "C" Products Grouping

REQ3	1	Provide alert notifications for aging products based on product age criteria and timeline parameters, customizable to specific member distributions	System to send email or notification to specific distribution after a product sits for a certain time period
REQ4	3	Implement batch tracking (inventory categorization by batch numbers) methodologies for better management of expiration dates of perishable inventory	Categorize similar product groups by batch numbers Ability to query, view, and print reports based on batch information.
REQ5	2	User friendly and intuitive interface for easy navigation and customer usability, ability to identify problems directly in system	“Quick Pick” customizable queries for quick access to specific reports or visual dashboards
REQ6	3	Linkage of inventory with customer sales data or ERP software to accurately track incoming and outgoing material using the First-in-First-out method (Šimek, 2022)	After an "active ship" is made the system will remove inventory from "current stock" and send notification to key identified members
REQ7	4	Prioritize the sale of older or aging inventory through discounts or promotional strategies	Send notification to sales systems via email or linkage to sales ERP system to discount potential items based on age of item
REQ8	2	Provide detailed reports on aging stock by age brackets (i. 30, 60, 90 days)	Ability to print full reports based on age of stock
REQ9	1	Provide real-time access to stock levels and age across various locations	Userface properly accepts queries to review total stock levels

Enumerated Non-Functional Requirements

Identifier	Priority	Requirement	Test Case
Functionality			
NF REQ10	4	The system must provide an interface that allows the user to customize the type of inventory stored and specify the acceptable length of time for holding the inventory.	Test the system with the insertion of different types of variable inventory.
NF REQ11	4	The system must have a security protocol to manage specific content that the user can access to ensure sensitive information is protected.	Test the system by having unauthorized user access sensitive information.
NF REQ12	3	The system should have seamless Integration with other business systems to minimize errors that are caused by data syncing issues.	Test the system by integrating the inventory system with current business systems.
NF REQ13	3	The system should allow for the creation of new user accounts with specific privileges.	Test the system by creating multiple user accounts.
NF REQ14	3	The system should allow for the user to add different suppliers.	Test the system by adding multiple suppliers to the system.
NF REQ15	2	The system could have the ability to compare different supplier inventory rates.	Test the system by having the system retrieve different suppliers and compare their prices.
Useability			
NF REQ16	4	The system must include a step-by-step process for installing software and setup.	Test the system with an inexperienced user and how the installation process goes without input from a technician.
NF REQ17	3	The system should be able to accommodate different sizes of businesses, sizes of inventory, and types of inventories.	Test the system by increasingly adding more inventory to see if usability deteriorates.

NF REQ18	3	The system should have a feedback mechanism for users to report issues or provide suggestions for improvement.	Test the system with feedback from the user to ensure that the feedback is going to the write technician.
NF REQ19	2	The system could have syncing capabilities across different networks of computers to be accessed anywhere within the company.	Test the system with new inventory information on one specific computer, then access the inventory list on a different computer to ensure that the inventory is updated correctly.
NF REQ 20	2	The system could offer different avenues the inventory could be re-marketed or distributed.	Test the system by seeing if the system can pull data from the web of vendors who would fit the criteria of the inventory.
Reliability			
NF REQ 21	3	The system should create multiple backups of inventory in case of system failure.	Test the system by introducing fatal crashes and see if the backup reports are still available.
NF REQ 22	4	The system must ensure that the inventory records match the physical stock by cross-referencing historical data, business supplier data, and current data to alert discrepancies.	Test the system with periodic manual checks of inventory to ensure that the system has accurate data.
Performance			
NF REQ 23	2	The system could support a high number of concurrent users, especially in large organizations without experiencing degradation in performance.	Test the system by having multiple users access inventory within the same time period to determine acceptable loss in performance without the system deteriorating.
NF REQ 24	2	The system could have a large number of inventory transactions experiencing degradation in performance.	Test the system by increasingly adding large number of inventory transactions to the systems to determine acceptable loss in performance without the system deteriorating
NF REQ 25	3	The user should not have to wait long periods of time for the software to initialize and be ready to use.	Test the system by ensuring the system does not take more than five seconds to boot up.
NF REQ 26	4	The system must be able to update inventory immediately to allow for real time tracking.	Test the system by assuring that when inventory is added to the system the system updates inventory instantly.
Supportability			
NF REQ 27	3	The system software should be designed in a modular way, making it easier to update or replace individual components	Test the system by ensuring that each individual module can run independently.
NF REQ 28	4	The system must offer training modules, training tutorials or guides to assist employees in understanding the functionalities.	Test the system with novice users and provide them with system documentation. Then ask the user to accomplish tasks that have been stated in the documentation.

User Interface Requirements

1. Dashboard Overview

Purpose: Provides users with a quick overview of inventory status, order tracking, and alerts. **User Interaction:**

- Users log in and land on a dashboard displaying real-time updates.
- Widgets display key statistics such as stock levels, pending orders, and alerts.
- Users can customize filters to view only relevant data.

2. Inventory Management Interface

Purpose: Allows users to add, edit, and track inventory items.

User Interaction:

- Users can search for items using filters (SKU, category, expiration date, etc.).
- Clicking an item opens a detailed panel where users can update stock, change location, or mark items as damaged.
- Batch actions allow updating multiple items simultaneously.

3. Customizable Customer Profile

Purpose: Allows customers to track their own inventory and receive alerts.

User Interaction:

- Users can create a profile and add their own items to their inventory.
- Option to set custom alerts for stock levels, expiration dates, and usage trends.
- The system provides inventory recommendations based on past trends.

4. Order Processing & Tracking

Purpose: Manages and fulfills customer orders efficiently.

User Interaction:

- Users can create new orders by selecting products and entering quantities.
- A real-time order tracking module shows pending, shipped, and delivered orders.
- Clicking on an order displays shipping details, customer info, and fulfillment status.

5. Alerts & Notifications

Purpose: Notifies users of urgent inventory events.

User Interaction:

- Pop-up alerts notify users of low stock, shipping delays, or system issues.
- A notification center stores past alerts for reference.

- Users can customize notification settings.

6. Recommendations & Guidelines

Purpose: Helps users optimize inventory management.

User Interaction:

- A dedicated section provides recommendations based on user data.
- Example: "You have 10 items expiring soon. Consider discounting or donating."
- Users can set rules for automated recommendations.

Functional Requirement Specification and Use Cases

The requirements, specifications, and use cases are outlined below in our diagram, traceability matrix, and fully dressed description. Below we have summarized how the project influences and directly affects unique stakeholders and actors.

Stakeholders

- *Business Owners:* have interest in reducing losses and maximizing profitability from obsolete or aging stock
- *Inventory Managers:* have interest in demand forecasting and identifying stagnant products and implementing rotation strategies
- *Warehouse Staff:* have an interest in operational productivity and storage efficiency such as prioritizing space and reducing time spent handling excess inventory
- *Procurement and Supply Chain Teams:* have an interest in mitigating supply chain disruptions and ensuring proper stock levels for ingoing and outgoing materials
- *Suppliers/Vendors:* have an interest in understanding product movement and product performance metrics
- *Consumers:* have an interest in fresher products and access to essential items as well as access to clearance sales and discounted aged inventory
- *Logistic Centers:* have an interest in ensuring aged stock is prioritized for shipping to avoid obsolescence
- *Supermarkets / Grocery Stores:* have an interest in using the system-to-be to manage a wide variety of inventory that includes perishable and nonperishable.
- *Employee Stocker:* have an interest in using the system-to-be to put inventory on shelves and find inventory.
- *Accountants:* have an interest in the system-to-be because they must understand the finances being spent on inventory.
- *IT and System Administration:* Responsible for maintaining and troubleshooting.

- *Retail Store Manager*: Need to monitor stock levels and make restocking decisions.
- *Quality Assurance Teams*: Ensure products meet quality standards before being sold or shipped.
- *Marketing and Sales Teams*: Use inventory data to plan promotions and optimize sales strategies.
- *Customer Support Teams*: Handle inquiries related to product availability and stock issues.
- *Compliance Officers*: Ensure the system adheres to the regulations and safety standards.

Actors and Goals

Actor	Participating / Initiating	Role	Goal
Accountant	Participating	The accountant's role is to use the reporting features to verify inventory by tracking the <u>movement of items</u> .	Goal is to identify and correct invoicing issues when items have shipped.
Administrator	Initiating	The administrator's role is to manage user accounts, specify account privileges and <u>specify who gets notified of inventory issues</u> .	The goal is to create new user accounts, specify privileges, and specify inventory alerts.
Email Notification service	Participating	The role of the email notification service is to push out email notification to specific users.	The goal is to identify when it is appropriate to send an email and send the <u>email to the correct user</u> .
Employee	Initiating	The employee's role is to be able to find available inventory within the company.	The goal is to search for specific inventory and be able to locate that inventory.
ERP Software	Participating	The ERP software's role is to manage all core operations from a centralized location.	The ERP software's goal is to link inventory with sales data.
Manager	Initiating	The manager's role is to distinguish types of inventory.	The goal is to retrieve inventory information that identifies whether items are <u>perishable or non-perishable</u> .
Manager	Initiating	The manager's role is to identify the shelf life of inventory.	The goal is to create unique customizable time for inventory.
Manager	Initiating	The manager's role is to determine how much shelf space for inventory.	The goal is to identify how much space has been allocated to specific inventory.
Manager	Initiating	The manager's role is to decide how much inventory needs to be ordered.	The goal is to retrieve the margin information to make a decision on how <u>much new inventory needs to be ordered</u> .
Marketing	Initiating	The role of marketing is to decide promotional strategies such as discounts the company should <u>take</u> .	The goal is to be able to identify aging inventory or popular inventory to make the <u>decisions</u> .
Third Party software	Participating	The role of the third party software is to provide unique data information about inventory.	The goal is to integrate data information into the system to have up to date <u>inventory specifications</u> .
Vendor	Participating	The vendor's role is to provide information of shipped inventory date, quantity, price, and arrival time.	The goal is to return accurate information on quantity and price on purchased inventory.

Use Cases

Casual Description

Database Customization:

Description: Customers can set item profiles for specific items to generate alerts based on criteria.

Responds to Requirements: REQ1, REQ2, REQ3, NF-REQ10, NF-REQ11, NF-REQ12, NF-REQ13, NF-REQ14, RN-REQ15, NF-REQ16, NF-REQ17, NF-REQ18, NF-REQ19, NF-REQ20, NF-REQ21, NF-REQ23, NF-REQ26, NF-REQ27, NF-REQ28.

Product to Vendor/Sales Linking:

Description: Items are ordered automatically based on a threshold the customer can set in the item profile to ensure consistent flow procuring items.

Responds to Requirements: REQ4, REQ6, NF-REQ10, NF-REQ12, NF-REQ15, NF-REQ17, NF-REQ18, NF-REQ19, NF-REQ20, NF-REQ21, NF-REQ22, NF-REQ24, NF-REQ25.

Categorize Products by Age:

Description: Labels for items generated based on user specific criteria, date item was received, and current day.

Responds to Requirements: REQ1, REQ2, REQ4, REQ5, REQ6, REQ7, REQ8, REQ9, NF-REQ10, NF-REQ12, NF-REQ17, NF-REQ18, NF-REQ19, NF-REQ20, NF-REQ21, NF-REQ27.

Alerts for Aging Items:

Description: Notification to specific groups that items are perishing or have been put-away and not shipped for specific amounts of time set by the user.

Responds to Requirements: REQ1, REQ2, REQ3, REQ4, REQ7, REQ8, NF-REQ10, NF-REQ12, NF-REQ17, NF-REQ18, NF-REQ19, NF-REQ20, NF-REQ21, NF-REQ22, NF-REQ23.

Searching through the Database:

Description: Customers are enabled to search through the database for specific items or purchase orders.

Responds to Requirements: REQ4, REQ5, REQ6, REQ9

Logging into the Database:

Description: Customers are enabled to sign into the database, protecting their data from those who should not have access.

Responds to Requirements: NF-REQ11, NF-REQ12, NF-REQ13

Demand Forecasts:

Description: Users can generate a report that shows patterns of demand. For instance how much inventory was shipped and when. Enabling the customer to plan for low demand or a need for excess stock.

Responds to Requirements: REQ6

Optimize retail prices:

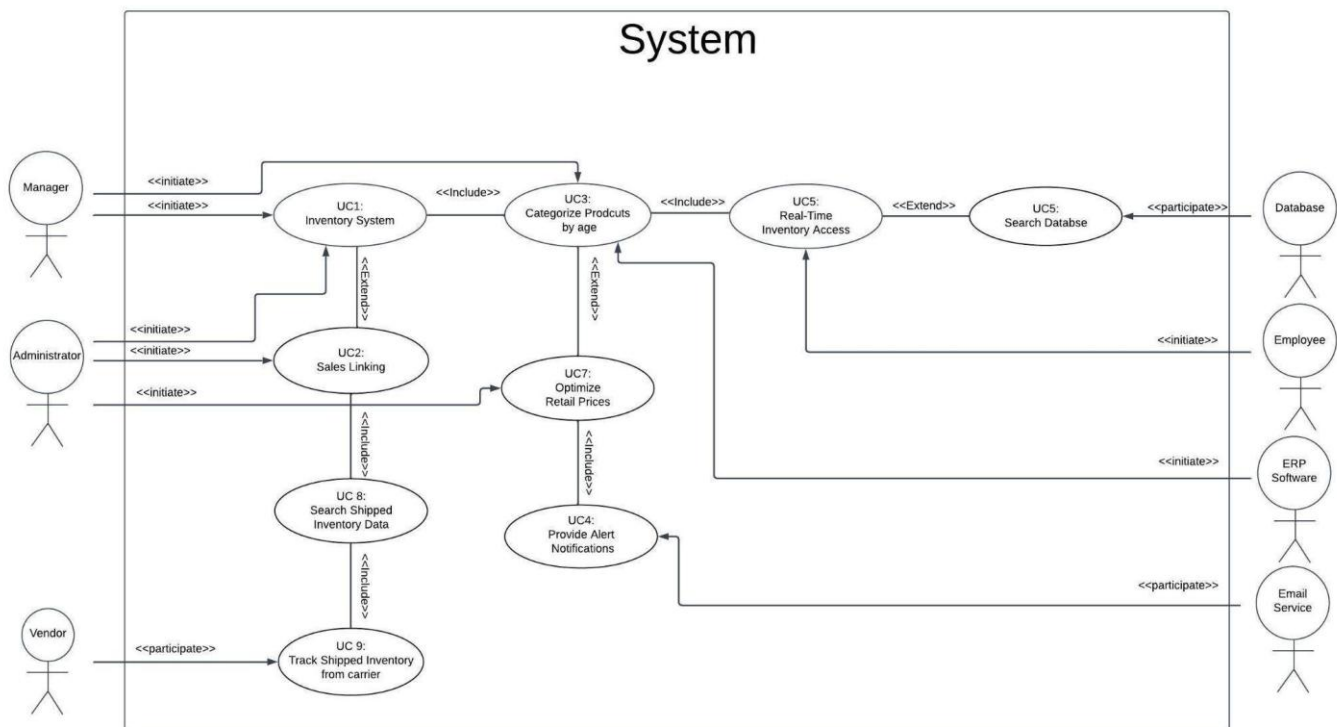
Description: System identifies slow moving items and creates recommendations to the user such as discounts, bundle pricing, and competitive pricing.

Responds to Requirements: REQ2, REQ7

Average inventory cost:

Description: System provides an average cost for items over a set time period taking into account the fluctuations of price depending on season, shipping, and labor.

Responds to Requirements: REQ2, REQ6

Use Case Diagram

You can view the full WasteLesss use case diagram using the following link:

https://lucid.app/lucidchart/26f6dc7c-b493-44aa-aa5f-872fa2a4fb67/edit?viewport_loc=-2270%2C-653%2C3827%2C2165%2C0_0&invitationId=inv_8a5fd375-4b4b-4880-9456-2c1dbb5c7e57

System Components

- **Inventory**
 - Tracks all products in the warehouse.
 - Attributes: Item ID, Name, SKU, Batch Number, Category, Expiration Date, Received Date, Current Status (Received, Putaway, Allocated, Shipped).
- **Orders**
 - Keep track of orders placed for inventory allocation.
 - Attributes: Order ID, Customer ID, Order Date, Status (Pending, Shipped, Delivered), Items Ordered.
- **Shipping Carrier**
 - Manages shipping logistics for outgoing inventory.
 - Attributes: Carrier Name, Tracking Number, Estimated Delivery Date.
- **Warehouse Locations**
 - Defines storage locations within the warehouse.
 - Attributes: Location ID, Aisle, Bin Number, Storage Type (Reserve/Active).
- **Aging Inventory Tracking**
 - Focuses on identifying aging inventory and waste.
 - Attributes: Item ID, Received Date, Current Age, Aging Status (Fresh, Near Expiry, Overstocked).
- **Customers & Customer Profiles**
 - Tracks customer details and preferences.
 - Attributes: Customer ID, Name, Contact Information, Order History, Preferred Inventory Categories.
- **Vendors**
 - Maintains supplier information for inventory restocking.
 - Attributes: Vendor ID, Name, Contact Information, Supplied Products.
- **Alerts & Notifications**
 - Manages system-generated alerts for various inventory issues.
 - Attributes: Alert Type, Affected Item ID, Date Triggered, Assigned Department.
- **Company Divisions**
 - Helps categorize company departments responsible for inventory decisions.
 - Attributes: Division ID, Name (e.g., Inventory Control, Marketing, Allocation, Returns).
- **Item Pack Type**
 - Defines packaging types for better inventory tracking.
 - Attributes: Pack ID, Description
- **Net Promoter Score**
 - Tracks customer feedback on system efficiency.
 - Attributes: Score, Customer ID, Review Comments.

Traceability Matrix

Req's	PW	UC1	UC2	UC3	UC4	UC5	UC6	UC7
Req 1	3	X						
Req 2	4	X		X				X
Req 3	1				X			
Req 4	3		X					
Req 5	2					X		
Req 6	3	X	X				X	
Req 7	4				X			X
Req 8	2			X	X			
Req 9	1					X		
Max PW		4	3	2	4	2	3	4
Total PW		10	6	6	7	3	3	8

Fully Dressed Descriptions

Use Case UC-1:	Database Customization
Related Reqs:	REQ-1, NF REQ-14
Initiating Actor:	User [Inventory Manager: Name]
Actor's Goal:	Customize Inventory management system settings to fit specific business needs.
Participating Actors:	System Admin, System User

Preconditions:	User is logged into the system with administrative privileges. System is operational and connected to database
Postconditions:	Customization settings are successfully applied The system reflects the changes in relevant modules
Flow of Events for Main Success Scenario:	
→ 1.	The Inventory Manager accesses the system
← 2.	System displays available customization options via different tabs
→ 3.	The user selects the tab for which customization option they want to modify
← 4.	The system prompts for input (category, product, perishable/nonperishable, age category, vendor name, contact, address, preferred status, age categories and thresholds.
→ 5.	The user enters customization details and submits
← 6.	The system validates the input and applies the new settings
← 7.	Confirmation message is displayed and changes are logged
Flow of Events for Extensions (Alternative Scenarios):	
← 7.	Saving Error - Error message is displayed and changes are not logged

Use Case UC-2:	Product to Vendor/Sales Linking
Related Reqs:	REQ, REQ 4, REQ 6
Initiating Actor:	User [Inventory Manager: Name]
Actor's Goal:	Associate a product with a vendor profile for procurement and sales tracking
Participating Actors:	Vendor (External Supplier), Sales Team, Optional: ERP Software
Preconditions:	The product is created and exists in the inventory database

	The vendor details are accessible in the system User has necessary permissions to modify product-vendor relationships
Postconditions:	The product is successfully linked to a vendor and available for sales and restocking The system updates procurement and sales records accordingly
Flow of Events for Main Success Scenario:	
→ 1.	The inventory manager navigates to the product linking module
← 2.	System displays available products
→ 3.	The user selects a product to link
← 4.	The system fetches a list of available vendors
→ 5.	The user selects a vendor from the list or adds a new one
← 6.	System prompts the user to enter linking details (pricing, contract terms, lead times)
→ 7.	User submits linking request
← 8.	The system validates the input and applies the new settings
← 9.	Confirmation message is displayed and changes are logged
Flow of Events for Extensions (Alternative Scenarios):	
→ 5.	Vendor not found - If the desired vendor is not in the system, the user is prompted to add a new vendor before proceeding
→ 7.	Missing Data - If required fields are missing the system requests corrections before proceeding
← 8.	Connection Error - If the system cannot save the linking data, the system notifies the user and logs an error
Use Case UC-3:	Categorize Products by Age

Related Reqs:	REQ 2, REQ 8
Initiating Actor:	System, User [Inventory Manager: Name]
Actor's Goal:	Classify products based on their age using predefined aging criteria
Participating Actors:	Inventory Database, Reporting and Analytics Module
Preconditions:	The system has predefined age categories (“new”, “aging-soon”, “expired”, “ready for resale”) Products have timestamps indicating when they were received Aging criteria and threshold are configured in the system
Postconditions:	Each product is assigned an appropriate aging category Categorization data is available for reporting, notification, and stock management decisions
Flow of Events for Main Success Scenario:	
← 1.	The system runs a scheduled check on all inventory products
← 2.	The system retrieves aging criteria and categorization rules
← 3.	The system checks each product’s age based on its timestamp
← 4.	The system assigns the product to an appropriate category (“new”, “aging-soon”, “expired”, “ready for resale”)
← 5.	The system updates the product record in inventory database with the new category
← 6.	The system generates reports and updates dashboards
→ 7.	The Inventory Manager can manually review and override categorizations if needed
Flow of Events for Extensions (Alternative Scenarios):	
← 4.	No Matching Category - if a product’s age does not fit into any predefined category, they system logs an error and requests manual override

← 6.	Connection Error - If the system cannot save the linking data, the system notifies the user and logs an error
→ 7.	Manual Override - Inventory Manager selects a product and manually changes its age category and system logs changes

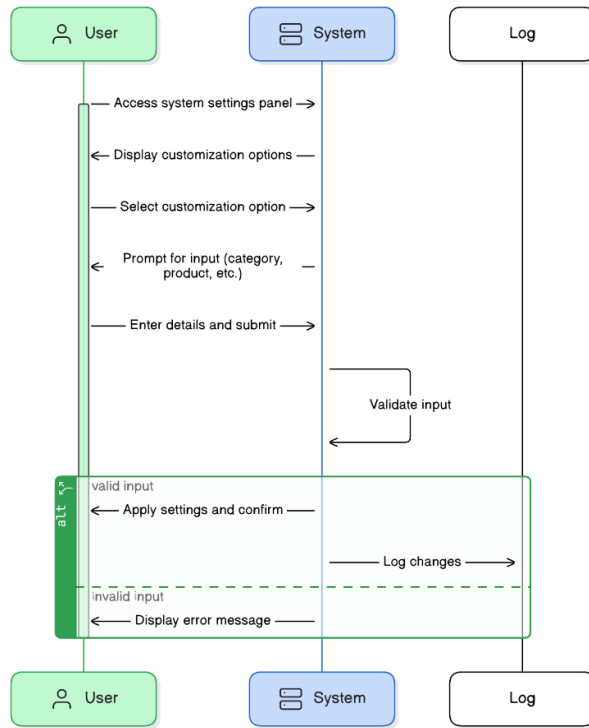
Use Case UC-4:	Provide Alert Notifications for Aging Products
Related Reqs:	REQ 3
Initiating Actor:	System [Inventory Manager: Name]
Actor's Goal:	Ensure aging products are flagged and notifications are sent to the appropriate distribution members based on predefined criteria
Participating Actors:	User, Distribution Members, Email/Notification System, Inventory Database
Preconditions:	The product aging criteria and timeline parameters are configured in the system System has access to product inventory records, including timestamps for added products Distribution members are assigned and have up-to-date contact information
Postconditions:	Notifications are successfully sent to designated distribution members System logs the notification event Inventory record is updated with the alert status
Flow of Events for Main Success Scenario:	
← 1.	The system runs a scheduled check on inventory products
← 2.	The system retrieves the aging criteria and timeline parameters
← 3.	The system scans the inventory database to identify products that exceed the defined aging threshold
← 4.	The system generates an alert notification containing the aging status
← 5.	The system sends an email or system notification to the designated distribution members

← 6.	The system logs the notification event and updates the inventory record with the alert status
→ 7.	Distribution members receive the alert and take necessary actions (markdown, disposal, restocking strategy)
Flow of Events for Extensions (Alternative Scenarios):	
← 5.	Email / Notification Failure - if an email or notification fails to send, the system retries up to three times, if it still fails, system notifies the user and logs and error
→ 7.	Product Already Flagged - If a product has already been flagged as aging and a notification was previously sent, the system sends notification to Inventory Manager

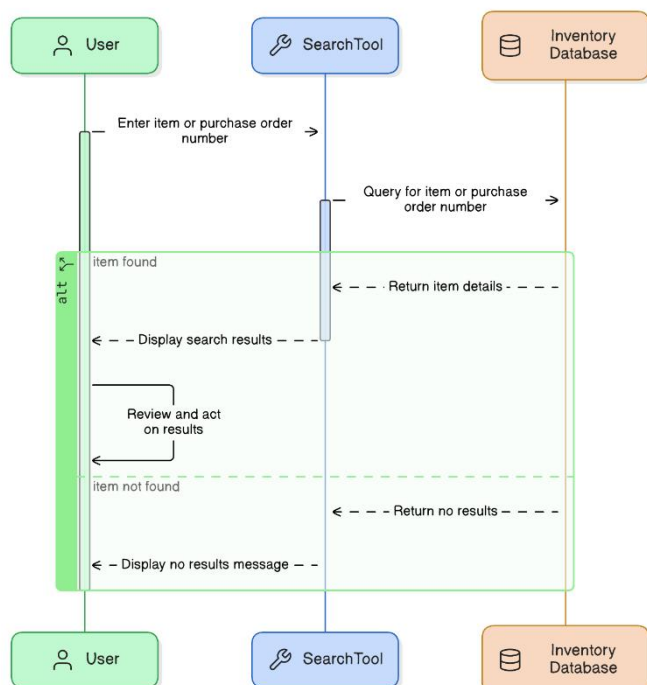
Use Case UC-5:	Searching through the Database
Related Reqs:	REQ 9
Initiating Actor:	User [User: Name]
Actor's Goal:	Using search tools to identify products within the database.
Participating Actors:	Inventory database
Preconditions:	The product is logged within the database System has access to all product variables within inventory records Employees have item or purchase order numbers they need more information on
Postconditions:	Results of the inventory search are displayed to the user
Flow of Events for Main Success Scenario:	
→ 1.	The user enters an item or purchase order number into the search tool
← 2.	The system retrieves results for the specific item or purchase order number
→ 3.	The user reviews and acts on the results from the search

System Sequence Diagrams

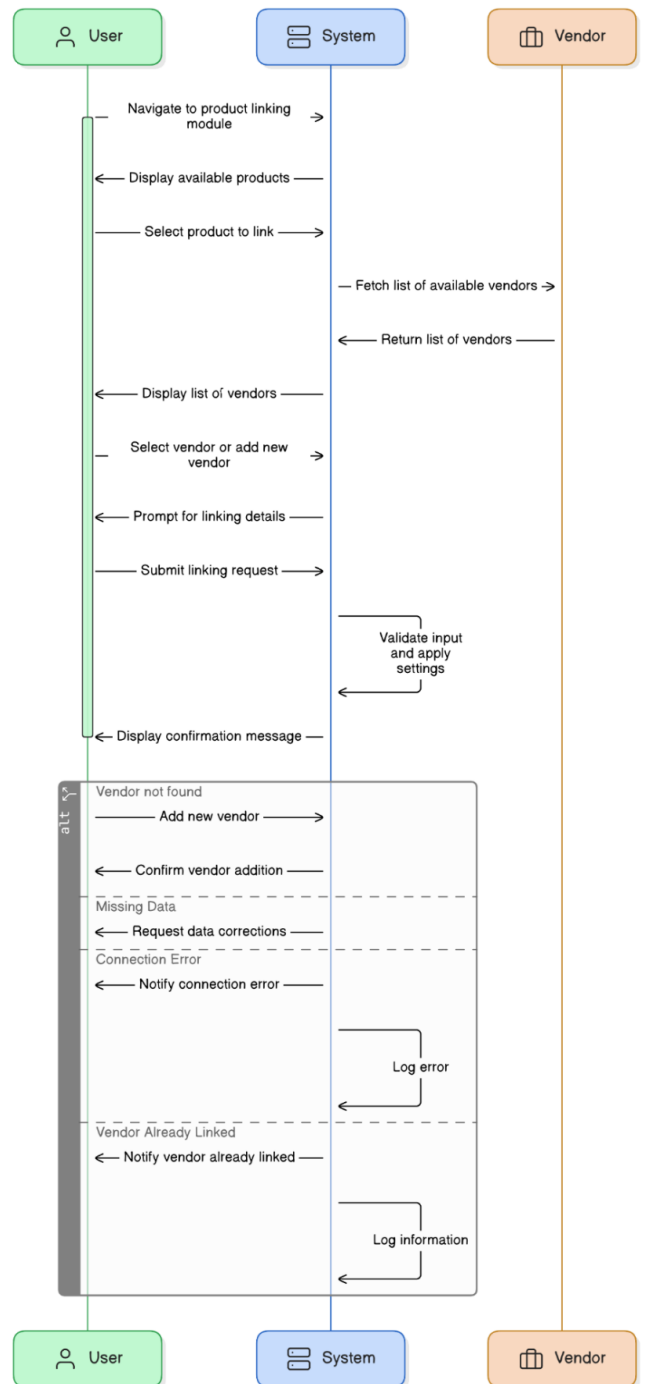
Inventory Customization Process



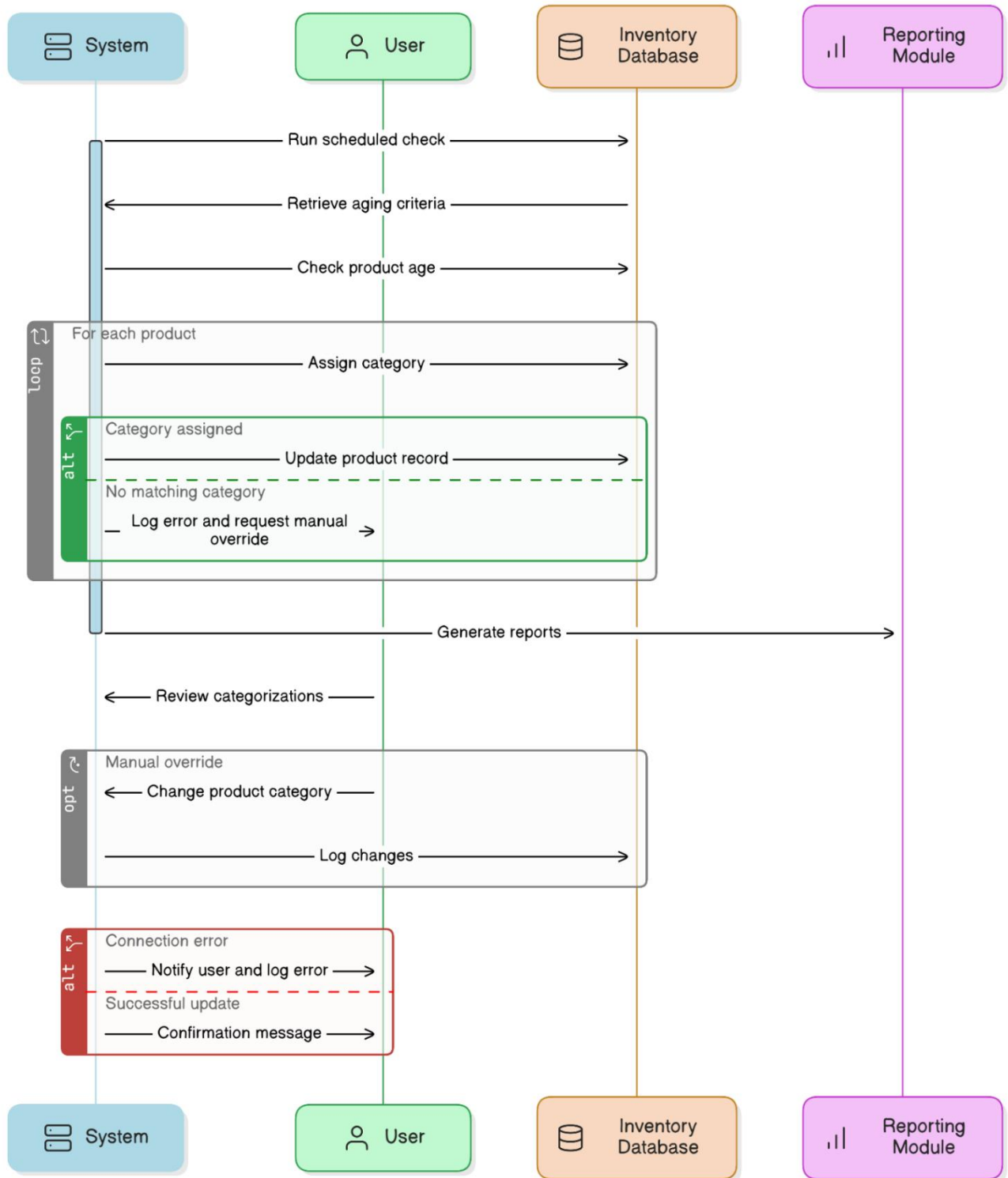
Searching through the Database



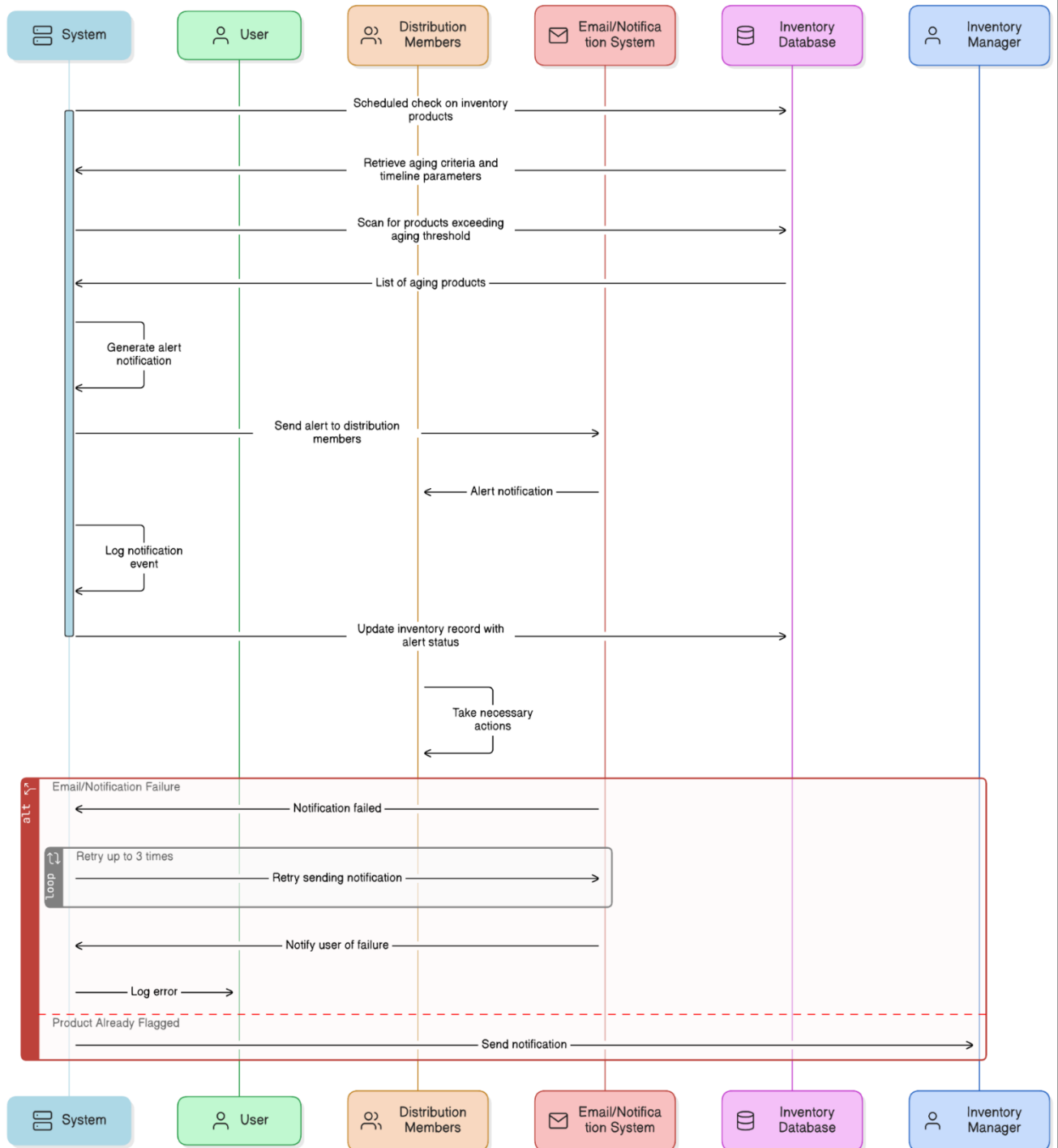
Product to Vendor/Sales Linking



Categorize Products by Age



Provide Alert Notifications for Aging Products



User Interface Specification

1. Logging In & Accessing the Dashboard

- Users enter credentials (email & password) on the login screen.
- Upon successful login, the dashboard appears, showing key inventory statistics and alerts.

2. Adding a New Inventory Item

Steps:

1. Navigate to **Inventory Management**.
2. Click "**Add New Item**".
3. Enter item details:
 - Item Name
 - SKU
 - Category
 - Quantity
 - Expiration Date (if applicable)
 - Storage Location
4. (Optional) Upload an image or scan a barcode.
5. Click "**Save**" to add the item.

Display Result:

- The new item appears in the inventory list.
- The system triggers alerts if stock levels are below the set threshold.

3. Setting Alerts for an Item

Steps:

1. Navigate to **Inventory Management**.
2. Select an item and open its details panel.
3. Click "**Set Alert**" and configure:
 - Low stock notification
 - Expiration date reminder
 - Overstock alert
4. Click "**Save**".

Display Result:

- Alerts appear in the notification center when triggered.
- Users receive notifications via email or app alerts.

4. Viewing Customer Profiles & Recommendations

Steps:

1. Navigate to **Customer Profiles**.
2. Search for a customer by name or ID.
3. View customer order history, preferences, and inventory.
4. Click "**View Recommendations**" to see inventory suggestions.

Display Result:

- A list of recommended actions appears (e.g., "Reorder stock soon").
- Users can take quick actions such as restocking or setting new alerts.

5. Placing an Order

Steps:

1. Navigate to **Order Management**.
2. Click "**New Order**".
3. Select products, enter quantities, and confirm details.
4. Choose a shipping method and finalize payment.
5. Click "**Place Order**".

Display Result:

- Order status updates in real-time.
- Users receive an order confirmation and tracking details.

6. Managing Warehouse Locations

Steps:

1. Navigate to **Warehouse Map**.
2. Search for a product.
3. View location (aisle/bin number) on the map.
4. Drag-and-drop to reassign location if needed.

Display Result:

- Updated product location is saved and reflected in inventory.

7. Reviewing Reports & Analytics

Steps:

1. Navigate to **Reports & Analytics**.
2. Select a report type (inventory trends, sales performance, etc.).
3. Customize filters for a specific date range or category.
4. Click "**Generate Report**".

Display Result:

- The system generates a visual report with charts and key metrics.
- Users can download or share the report.

Preliminary Design

The image below relates back to “Logging In & Accessing the Dashboard”:

The image shows a web browser window with a single tab titled "Aged Inventory Database". The browser's address bar contains a Google search icon and a "Finish update" button. The web application has a purple header bar with a "Menu" icon and the text "Log-in Screen". The main content area is gray and features a "Sign-in" form. The form has a purple header with the text "Sign-in" and two input fields: "Username*" and "Password*", both with red asterisks indicating required fields.

Sign-in	
Username*	
Password*	

The image below relates back to “Adding a New Item” and “Setting Alerts for an Item”:

Item profile

Purchase Order

Item Number

Perishable

☐

Non-Perishable

☐

Replen if <

units

Time interval alerts in days

1st

2nd

3rd

If perishable

Lost inventory

Aged inventory

Departments for alerts

☐ Inventory Team

☐ Operations Leadership

☐ Marketing Team

☐ Allocation Team

Reset

Submit

The image below relates back to “Setting Alerts for an Item” and “Viewing Customer Profiles & Recommendations”:

Department Personnel

Select department to edit

☐ Inventory Team

☐ Operations Leadership

☐ Marketing Team

☐ Allocations Team

Email to add user

Add

List of users within the department

Reset

Save

User Effort Estimation

$$\text{Estimated Effort (hours)} = \text{Use Case Points (UCP)} * \text{Productivity Factor (PF)}$$

PF = 28 hours per use case point

$$\text{Effort} = 47.06 * 28 = 1317.68 \text{ hours}$$

Method to find Use Case Points:

UCP = Technical Complexity Factor (TCF) * Environmental Factor (EF) * Unadjusted Use Case Weight (UUCW)

$$\text{UCP} = 1.04 * 0.905 * 50 = 47.06$$

Summary of Values	
UCP	47.06
UAW	11
UUCW	50
TCF	1.04
EF	0.905

To find the UAW of 11 we have the following actors and their weights:

$$\text{UAW} = 3+4+2+1+1+1 = 11$$

Actor	Type	Weight
Manager	Complex (Interactive User)	3
Administrator	Complex (Interactive User)	3
Employee	Average (Interactive User)	2
Vendor	Simple (Interactive User / System)	1
Database	Simple (System)	1
Email Service	Simple (External System)	1

To find the UUCW we have each use case and the basic number of steps (transactions):

$$\text{UUCW} = 15+10+10+5+10 = 50$$

Use Case	Type	Weight
UC1: Inventory System Customization	Complex	15
UC2: Sales Linking	Average	10
UC3: Categorize Products by Age	Average	10
UC4: Provide Alert Notifications	Simple	5
UC5: Real-Time Inventory Data Access	Average	10

Next we will use our rate of the 13 standard factors to find the TCF:

$$\text{TCF} = 0.6 + (0.01 * \sum (\text{rating} * \text{weight})) = 1.04$$

No.	Factor Description	Weight	Rating	Score
T1	Distributed system	2	4	8
T2	Response time/performance objectives	1	3	3
T3	End-user efficiency	1	4	4
T4	Complex internal processing	1	3	3
T5	Reusability	1	3	3
T6	Ease of installation	0.5	2	1
T7	Ease of use	0.5	2	1
T8	Portability	2	4	8
T9	Maintainability	1	3	3
T10	Concurrent use	1	3	3
T11	Security features	1	3	3
T12	Access for third parties	1	2	2
T13	Training needs for users	1	2	2
Total				44

Next we will use our ratings of the standard factors to find the EF.

$$\text{EF} = 1.4 + (-0.03 * \sum (\text{rating} * \text{weight})) = 0.905$$

Factor	Factor Description	Rating	Weight	Score
E1	Familiar with process	3	1.5	4.5
E2	App experience	2	0.5	1
E3	OO experience	2	1	3
E4	Analyst skill	3	0.5	2
E5	Motivation	5	1	5
E6	Stable requirements	2	2	4
E7	Part-time staff	3	-1	-1
E8	Difficult tech	3	-1	-2
Total				16.5

System Architecture and System Design

Identifying Subsystems

The system is divided into different subsystems, each handling a specific part of the inventory management process.

The **Inventory Management** subsystem keeps track of all products, including where they are stored, expiration dates, and stock levels. It helps make sure items are organized and available when needed.

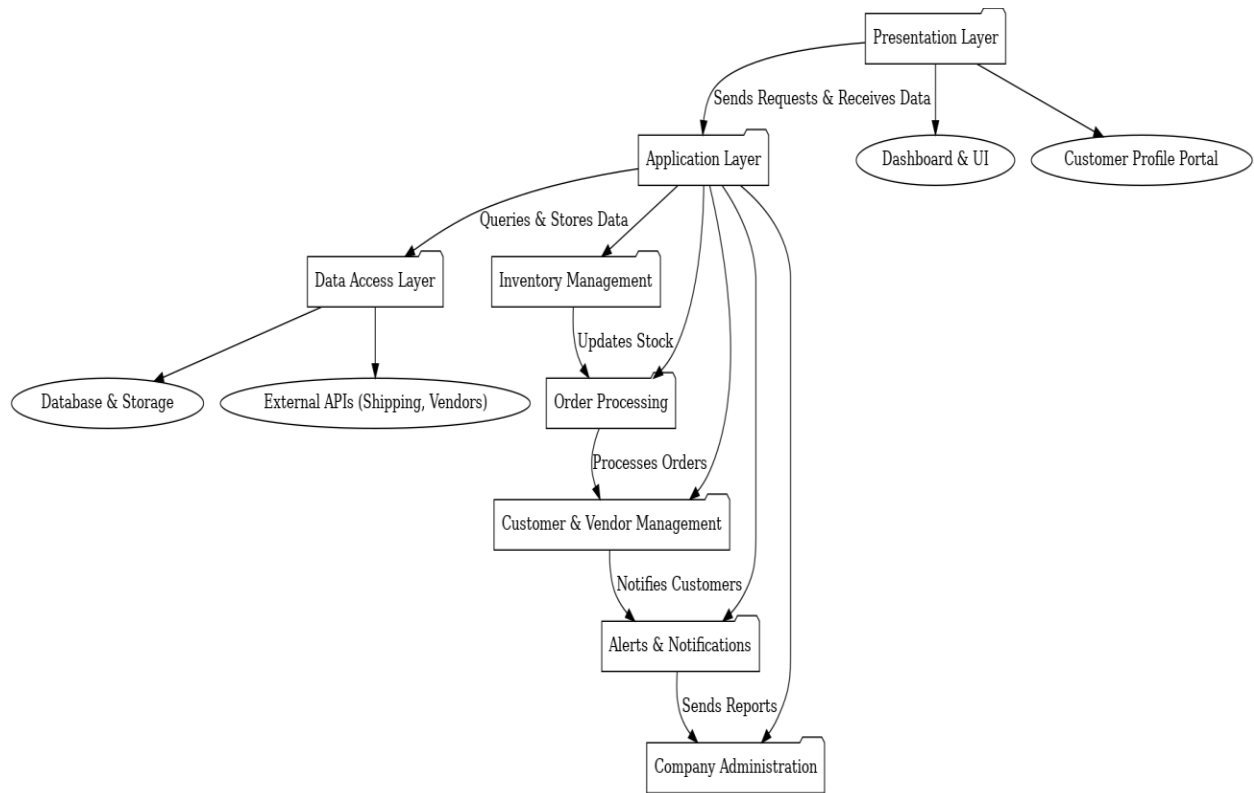
The **Order Processing** subsystem handles customer orders. It updates inventory when an order is placed and keeps track of shipping details to make sure orders are processed correctly.

The **Customer & Vendor Management** subsystem stores customer profiles and vendor details. It helps manage relationships, track past orders, and make reordering easier.

The **Alerts & Notifications** subsystem sends reminders and alerts about inventory issues. It notifies users when stock is running low, when products are about to expire, or when there are updates on orders.

The **Company Administration** subsystem manages business divisions and system-wide settings. It helps different departments track inventory and ensures that the system works efficiently across all areas of the business.

Each subsystem plays a role in making sure inventory is well-managed, orders are processed smoothly, and users stay informed about important updates.



Architecture Styles

The UML package diagram shows how the system is organized. It is divided into three main layers:

The **Presentation Layer**, also known as the client-side component and functionality, is what users interact with. Within this layer are sub-packages for different client modules such as where they can check inventory updates, manage orders, and set alerts. It also has a customer profile portal, where customers can track their own inventory, set reminders, and get recommendations.

The **Application Layer**, also known as the server side components, is where the main processes happen. Within this layer server related functionality services take place such as how inventory is tracked, orders are processed, and customer and vendor details are stored. It also handles alerts and notifications, making sure users are reminded about low stock, expiring products, or order updates. Company administration settings are also managed here.

The **Data Access Layer** is where all data is stored and retrieved. This layer functions as the communication or networking package where communication between the client and server takes place. Elements such as the database that holds inventory, orders, and

customer details. It also connects with external APIs to get real-time updates from shipping carriers and vendors.

All three layers work together combining the various components and subsystems within the architecture. When a user does something, like adding an item or placing an order, the presentation layer sends that request to the application layer, which processes it and pulls or updates data from the database. This setup keeps everything running smoothly, making the system easier to manage, update, and scale over time.

Mapping Subsystems to Hardware

Our system is designed to run on multiple computers with a distributed architecture.

Client (Web Browser) Machine: This machine accesses the subsystems such as Inventory Management, Order Processing, Customer and Vendor Management, Alerts and Notifications, and Company Administration via a web-based API

Server (Web/Application Server) Machine: This machine accesses the back-end services and runs the core application that processes requests from the client machine, manages inventory data and alerts, update orders, and customer/vendor records. This oversees the integration of subsystems.

Connectors and Network Protocols

Our system runs on multiple devices and requires communication between the front end, backend, and database. To handle this, we use the following communication protocols:

- **pg (Node-Postgres):** This is used for communication between our Node.js backend and PostgreSQL database. It allows us to store, retrieve, and update inventory, orders, and customer data efficiently through SQL queries.
- **HTTP/HTTPS (REST API):** This allows the frontend (user interface) to communicate with the backend (Node.js application). When a user requests inventory details or places an order, the frontend sends an HTTP request, and the backend processes it and sends back the data.
- **RESTful APIs:** These APIs manage the exchange of data between different parts of the system. For example, if a user wants to check inventory, the frontend sends a GET request, the backend fetches the data from the database using SQL, and the API returns the response in JSON format.

We chose the pg module because it is a lightweight and reliable way to connect Node.js applications to a PostgreSQL database. HTTP/HTTPS is widely used for secure communication

between web applications and servers. RESTful APIs allow structured and scalable data exchange, making it easy to integrate with future features like vendor or shipping services.

This setup ensures that our system can process user requests, update inventory in real-time, and handle orders efficiently.

Global Control Flow

Execution orderliness:

The system will be an event driven system, since the system-to-be will be reliant on the user performing some action and will send an event to some other service. The events that occur will be immutable, in order for the system to ensure a permanent record of the actions taken by the user as it regards to inventory. The events that trigger the system will be dependent on the user's needs, an event is triggered and a callback function will be performed. Triggering events would be real-time changes in stock levels, like when an item is sold or a new shipment arrives, triggering updates across the system to alert the user of inventory specifications like perishable or aged inventory.

Time dependency:

The system-to-be is a real-time event driven system that is time dependency for specific events. The system-to-be's triggering events must happen within a specific timeframe to ensure the smooth operation of the system. The software is periodic in the sense that it has regular intervals that update inventory being managed. The specific time constraints for each period can vary based on the business' needs and the type of inventory being managed. The JDBC communication between our Java backend and SQL database will be crucial to keeping an up-to-date inventory. The updating of real-time inventory must occur within seconds or milliseconds after a shipment has arrived to ensure that customers see accurate stock levels. Events that trigger alerts must also happen within seconds or milliseconds to ensure that the customer has accurate information of perishable or aged inventory. The system-to-be will also have periodic inventory checks to ensure accuracy. They can be done daily, weekly, monthly, or quarterly, depending on the customer's needs.

Hardware Requirements

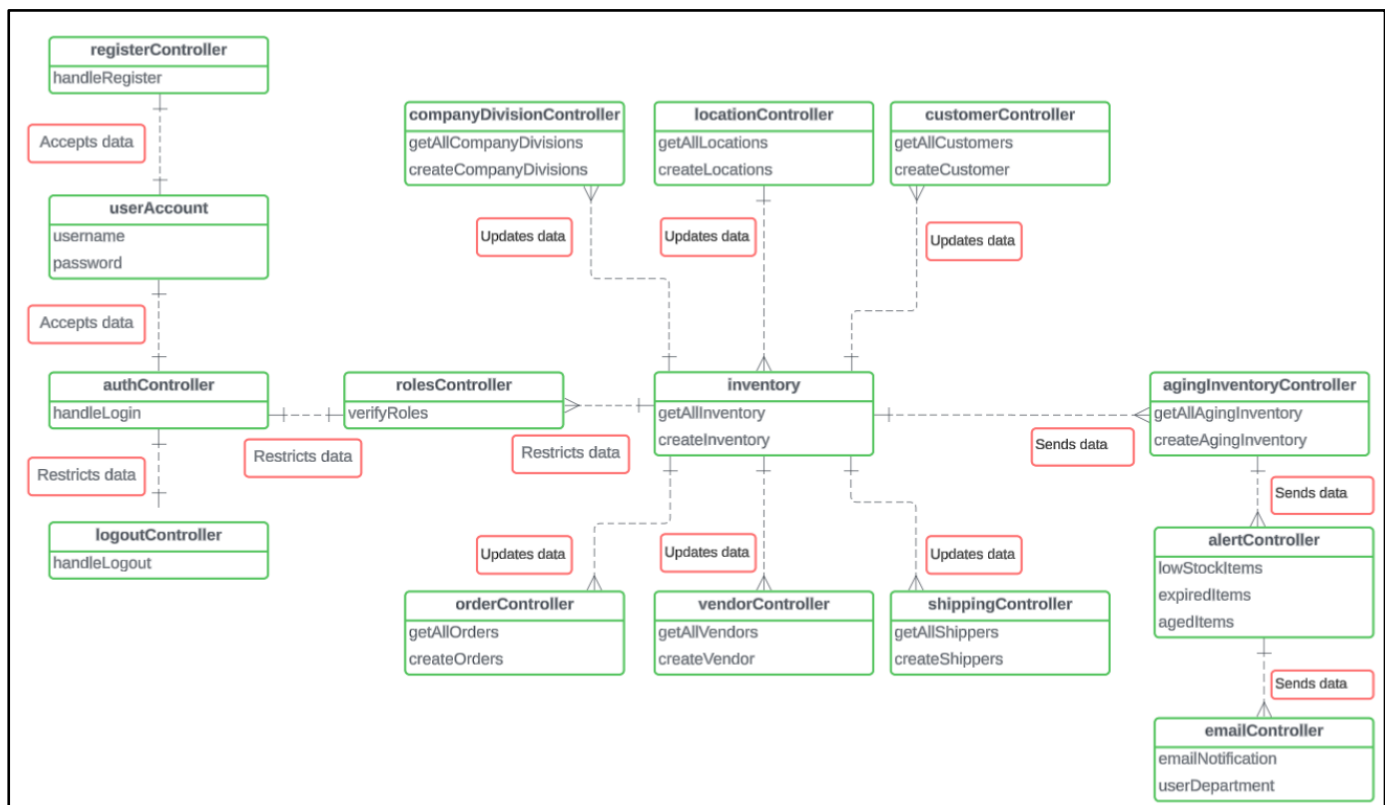
- **Processor:** The system requires a Central Processing Unit (CPU) for processing instructions,
 - Recommending a dual-core or quad-core, 2.0GHz or faster processor for best performance.
- **Memory:** The system requires Random Access Memory (RAM) to allow space for short term data created from applications.
 - Recommending 4GB or higher.

- **Screen Display:** The system requires an accessible output device to display the user interface. This interface can be accessed on various devices, including computers and phones.
 - Recommending minimum 640 x 480 pixels
- **Input Devices:** The system requires a keyboard for inputs and a mouse for user clicks.
- **Internet Modem and Router:** The system requires access to internet with a minimum speed of 1 Mbps.
 - Recommending 10 Mbps or higher

The intention is to create a system allowing any internet-enabled device to access the system using a database connection system, provided the user has the necessary permissions.

Analysis and Domain Modeling

Conceptual Model



Concept Definitions

Responsibility Description	Type	Concept Name
Enables user to create login information to access API.	D	registerController
Contains all the information for users to login to the API.	K	userAccount
Controls user access to the API.	D	authController
Enables user to logout, securing data and limiting access.	D	logoutController
Verifies the role and limits access appropriately.	D	rolesController
Enables user to get, update, create, and delete information in the inventory database.	D	inventory
Connects company division information to the inventory.	K	companyDivisionController
Connects location information to the inventory.	K	locationController
Connects customer information to the inventory.	K	customerController
Connects order information to the inventory.	K	orderController
Connects vendor information to the inventory.	K	vendorController
Connects shipping information to the inventory.	K	shippingController
Users set variables for specific items.	D	agingInventory
Controls alerts that are displayed to the user.	D	alertController
Formats and sends the notifications through email.	D	emailController

Association Definitions

Concept Pair	Association Description	Association Name
registerController↔userAccount	registerController submits username and password to userAccount	Accepts data
userAccount↔authController	userAccount submits user information to authController	Accepts data
authController↔logoutController	authController accepts user request to logoutController	Restricts data
authController↔rolesController	authController submits user information to rolesController	Restricts access
companyDivisionController↔inventory	companyDivisionController provides information for inventory.	Updates data
customerController↔inventory	customerController provides information for inventory.	Updates data
orderController↔inventory	orderController provides information for inventory.	Updates data
vendorController↔inventory	vendorController provides information for inventory.	Updates data
shippingController↔inventory	shippingController provides information for inventory.	Updates data

locationController↔inventory	locationController provides information for inventory.	Updates data
Inventory↔agingInventoryController	Inventory provides updates within data in real time to agingInventoryController	Provides data
agingInventoryController↔alertController	agingInventoryController sends data to the alertController	Provides data
alertController↔emailController	alertController calls the emailController to send notification via email.	Provides data

Attribute Definitions

Concept	Attributes	Attribute Definition
registerController	handleRegister	Enables the user to register to the database.
userAccount	Username	Login name for the user to access their account.
	Password	Password for the user to access their account.
authController	handleLogin	Restricts users from accessing database with incorrect credentials.
logoutController	handleLogout	Restricts user from accessing database upon users request.
rolesController	verifyRoles	Verifies the roles the user has to limit access to the database.
companyDivisionController	getAllCompanyDivisions	Displays all company division information to the user.
	createCompanyDivisions	Enables the user to create company division data.
locationController	getAllLocations	Displays all location information to the user.
	createLocations	Enables the user to create location data.
customerController	getAllCustomers	Displays all customer information to the user.
	createCustomers	Enables the user to create customer data.
orderController	getAllOrders	Displays all order information to the user.
	createOrders	Enables the user to create order data.
vendorController	getAllVendors	Displays all vendor information to the user.
	createVendor	Enables the user to create vendor data.
shippingController	getAllShippers	Displays all shipper information to the user.
	createShippers	Enables the user to create shipper data.
inventory	getAllInventory	Displays all inventory information to the user.
	createInventory	Enables the user to create inventory data.

agingInventoryController	getAllAgingInventory	Displays all aging inventory information to the user.
	createAgingInventory	Enables the user to create aging inventory data.
alertController	lowStockItems	Specific alert for low stock items.
	expiredItems	Specific alert for expired items.
	agedItems	Specific alert for aged items.
emailController	emailNotification	Email sent to the email address associated with the user selected for the specific alert.
	userDepartment	Email addresses associated with the specific department.

Traceability Matrix

Domain Concepts	Use Case	UC1	UC2	UC3	UC4	UC5
	PW	10	6	6	7	3
registerController		X	X			
userAccount		X	X			
authController		X	X			
logoutController		X	X			
rolesController		X	X			
inventory				X		X
companyDivisionController						X
locationController						X
customerController						X
orderController						X
vendorController						X
shippingController						X
agingInventoryController				X	X	
alertController					X	
emailController					X	

System Operations Contracts

Operation	Database Customization
Use Case	UC1

Preconditions	- User is able to log into the system with administrative privileges - The system is connected to the database and can process updates - User is able to input new customizable option - Customize options must meet input criteria
Postconditions	- The system displays relevant input fields for customization - Customization settings are successfully applied to the system - The system updates the relevant categories with new inputted items - An error message is displayed informing user of failure of input criteria, prompting to correct input

Operation	Product to Vendor/Sales Linking
Use Case	UC2
Preconditions	- The user is logged into the system and has permissions to modify product-vendor relationships - The product is created and exists in the inventory database and is displayed on the interface - Vendor is active and vendor details are accessible in the system - System is online and able to process linking requests - The product is not already linked to the same vendor (unless updating details)
Postconditions	- The product is successfully linked to a vendor and is available for sales / procurement - The system updates procurement and sales records accordingly - Vendor and product details are correctly associated in the inventory database - Notifications are sent to relevant departments (Sales, Procurement etc.) - The system logs all modifications for tracking and auditing purposes - An error message is displayed informing user if a product is already linked to the same vendor, or if vendor isn't active

Operation	Categorize Products by Age
Use Case	UC3

Preconditions	- The system has predefined age categories (“new”, “aging-soon”, “expired”, “ready for resale”), defined in UC1. - Products have been timestamped - Aging criteria and thresholds are configured and active in the system - Inventory data is accessible and up to date. - Dashboard is available and viewable
Postconditions	- Each product is assigned an appropriate aging category based on system rules - Categorization data is stored in the inventory database for tracking and analysis. - The system updates dashboards with the latest categorization details - Notifications are sent if age thresholds are triggered (expiring products, slow moving product) - The system logs all changes for tracking and auditing purposes - An error message is displayed if an error occurs such as missing timestamps, system error.

Operation	Provide Alert Notifications for Aging Products
Use Case	UC4
Preconditions	- The product aging criteria and timeline parameters are properly configured in the system. - The system has access to product inventory records, including timestamps for when products are received and expiring - The notification system is functional and integrated with the inventory database - Distribution members are assigned to receive alerts and have up-to-date contact information - The system can perform automatic data review to provide alerts during certain thresholds
Postconditions	- Notifications are successfully sent to designated distribution members - Inventory records are updated with the alert status - Distribution members receive alerts and take appropriate actions - The system logs all changes for tracking and auditing purposes - If notifications fail, retry attempted, and notification sent to inventory manager

Operation	Searching through the Database
Use Case	UC5
Preconditions	- The product is logged within the database - The system has access to all product variables within the inventory records - Employees have access to the system and is logged in to search - The search functionality is operational and properly configured
Postconditions	- Results of the inventory search are displayed for the user - If no matching results are found, the system provides appropriate feedback to the user - The system logs the search query for tracking and auditing purposes

Data Model and Persistent Data Storage

The system to be will need to save data that will outlive a single execution of the system. Persistent objects that will outlive single executions include vendors, customers, Inventory, location, orders, shopping, and company division. The system-to-be will use relational databases to store persistent data. The relationship database storage management strategy offers, data integrity and accuracy which our system-to-be is heavily dependent on.

The system will utilize PostgreSQL as the relational database model system and all the database capabilities needed by our application. Barker's notation in the representation of our diagram to who appropriate relationships between entities.

Entity

- Information that must be tracked, Person, Place, Thing, Concept, Organization, Event

Prime (Exists independently)

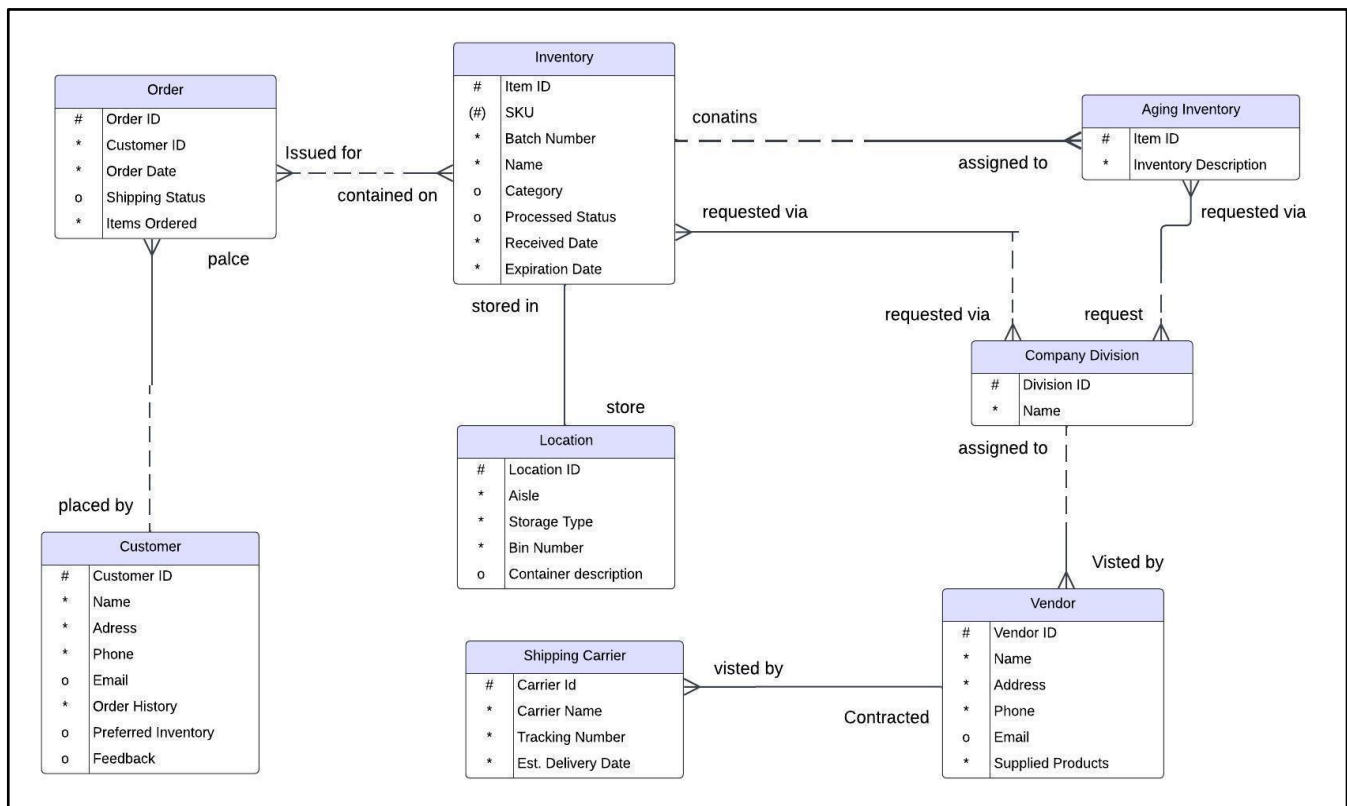
- Inventory
- Location
- Vendor
- Customer
- Company Division

Characteristic (Exists because of another(prime) entity)

- Order
- Aging Inventory
- Shipping Information

Relationship

- Inventory may contain one or many Aging Inventory
- Aging Inventory must be assigned to one and only one Inventory
- Inventory must be stored one and only one location
- A location must store one and only one Inventory
- An order may be issued for one or more inventory
- Inventory must be contained on one or more orders
- A customer may place one or many orders
- An order must be placed by one and only one customer
- A vendor must be visited by a company division
- A company division may be assigned to a vendor
- A vendor must contract one more or many shipping carriers
- A shipping must be visited by a vendor
- Aged Inventory must be requested via one or more company division
- A company division may request one or more aged inventory
- Inventory must be requested via one or more company division
- A company division may request one or more inventory



The full diagram can be found using the following link: https://lucid.app/lucidchart/d9b370ed-866b-412d-ba2c-a86a8d6692e3/edit?viewport_loc=-2658%2C-558%2C2574%2C1457%2C0_0&invitationId=inv_04205cb5-1977-4596-a600-fa7b8162332f

Mathematical Model

Basic Mathematical Model for Inventory Optimization

Our inventory system uses mathematical formulas to automate alerts and optimize inventory control. These models help reduce waste, prevent stockouts, and support data-driven decisions.

1. Economic Order Quantity (EOQ) Model

The EOQ formula helps determine the ideal quantity to reorder, balancing the cost of placing orders with the cost of holding inventory:

EOQ Formula:

$$\sqrt{\frac{2DS}{H}} \quad \text{EOQ} =$$

Where:

- **D** = Annual demand (units per year)
- **S** = Cost to place a single order
- **H** = Annual holding cost per unit

This calculation is used in the backend to automatically suggest reorder quantities when stock falls below the reorder threshold.

EOQ Calculation in Practice (Node.js Implementation)

In the `runAllAlerts()` function in `alertController.js`, our backend uses the EOQ formula to calculate reorder suggestions when stock is low. If demand, ordering cost, and holding cost are available for an item, the system calculates EOQ and includes a recommendation in the alert:

⚠ Printer ink stock is low! We recommend ordering about 15 units.

If the necessary values are missing, the alert is still created without a reorder recommendation.

2. Low Stock Alert Condition

We define a threshold for each item. When the stock level drops below this threshold, the system triggers a low stock alert:

Alert = true if `currentStock < reorderThreshold`

This condition is checked each time the inventory is updated or when the alert check is triggered manually or automatically.

3. Expiration Alert Condition

For perishable items, the system checks whether the expiration date has passed or is approaching:

$$\text{Alert} = \text{true if } \text{expirationDate} \leq \text{currentDate}$$

This helps ensure expired items are flagged for removal before they are used or shipped.

4. Stock Availability Check (Order Processing) -

This logic is planned for future implementation.

Before confirming an order, the system verifies whether there is enough stock to fulfill it:

$$\sum (\text{items in order}) \leq \text{available stock}$$

If the condition is not met, the order is either placed on hold or split, and the manager is notified.

Summary of EOQ Benefits to the System

The EOQ formula helps our system determine the ideal amount of inventory to reorder when stock runs low. This prevents overstocking (which wastes space and money) and understocking (which can lead to missed sales or delays). By balancing ordering and storage costs, the system supports smarter purchasing decisions and helps businesses reduce waste and manage inventory more efficiently.

Interaction Diagrams

Key

Design Principle:

Identifications in Yellow

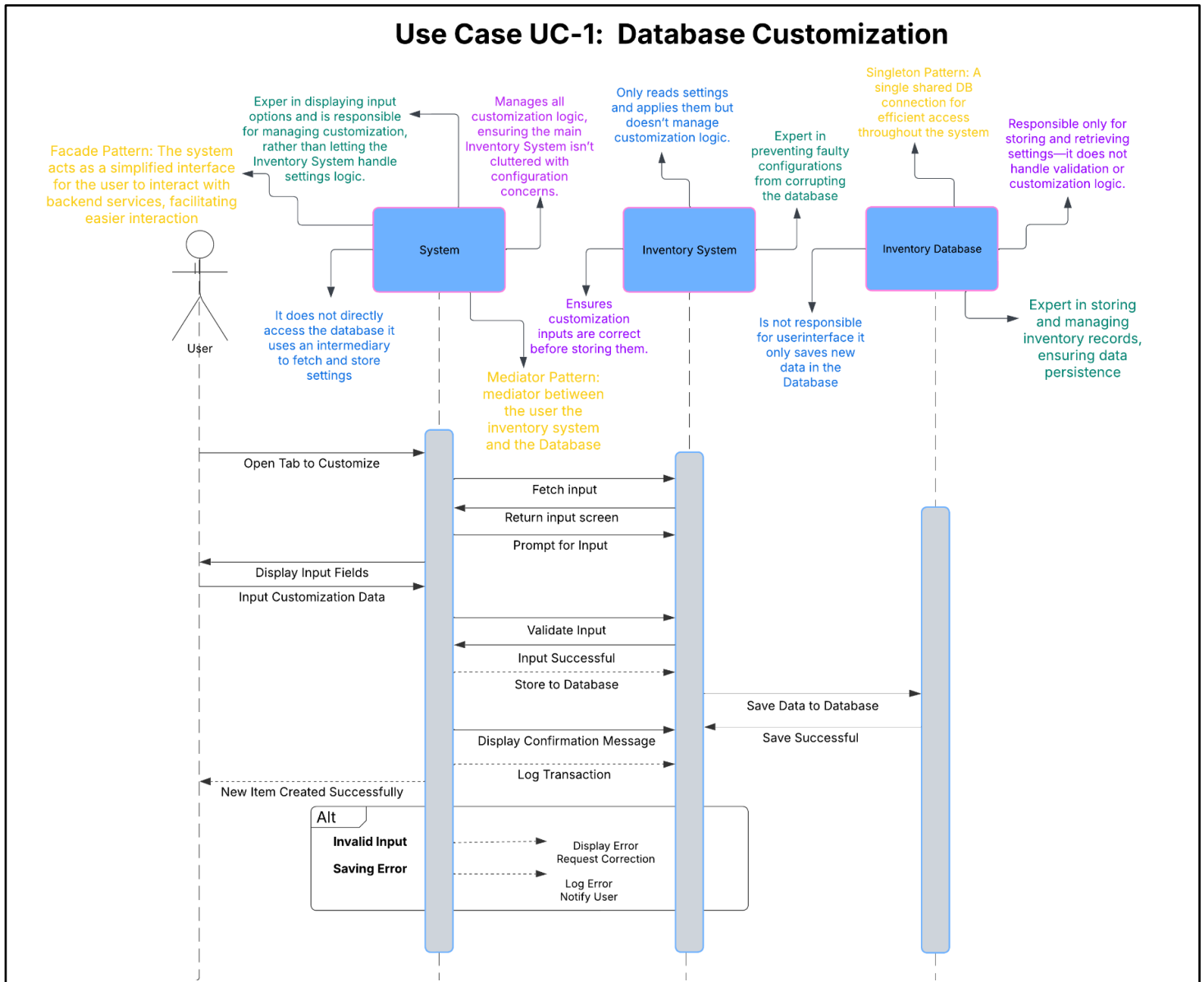
High Cohesion Principle:

Ensure each module has a clear focused responsibility
Responsibility Assignments in Purple

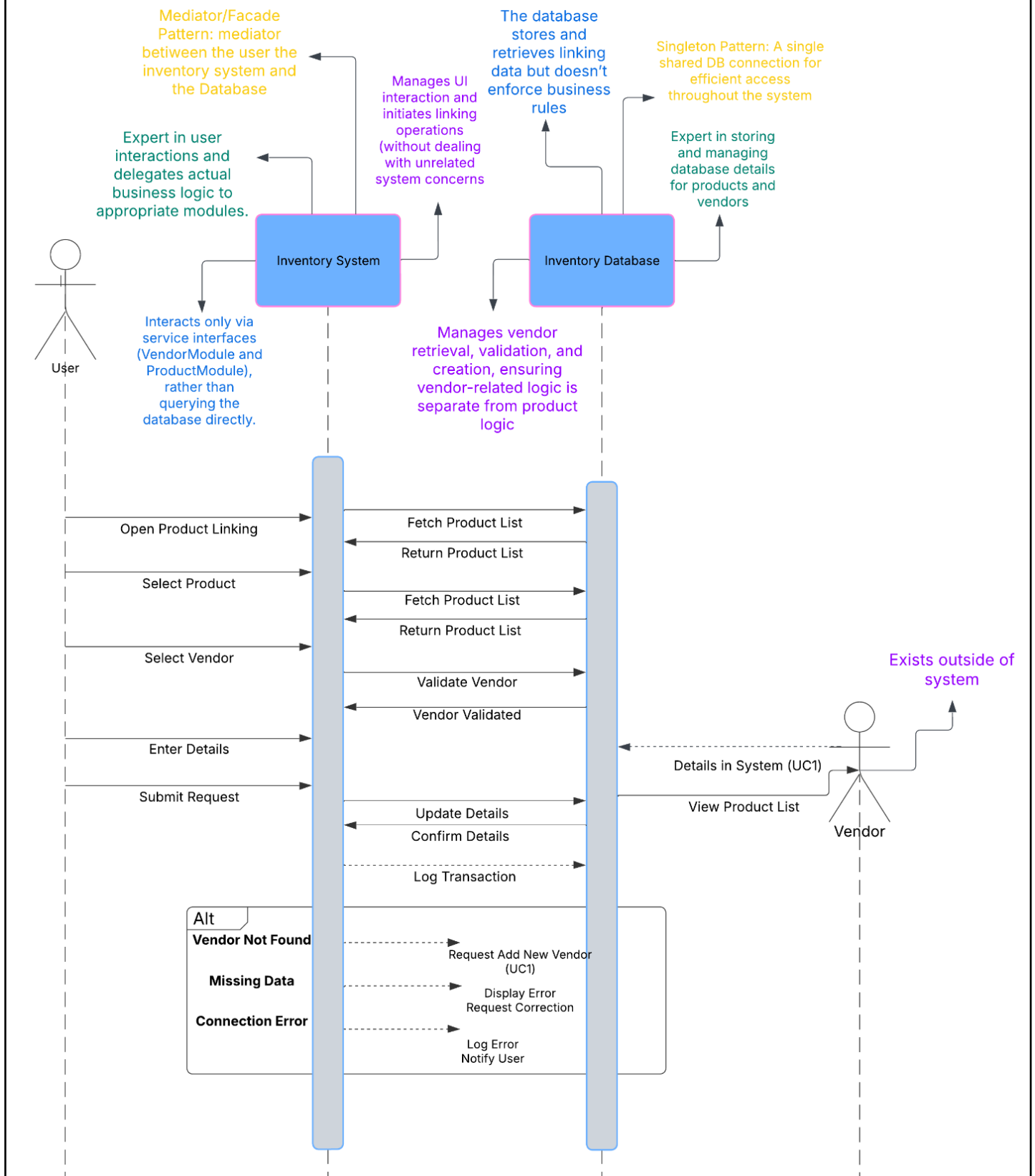
Low Coupling Principle: Minimize dependencies between modules to improve flexibility and maintainability
Responsibility Assignments in Blue

Expert Doer Principle: Assign responsibilities to the component that has the necessary data and expertise
Responsibility Assignments in Green

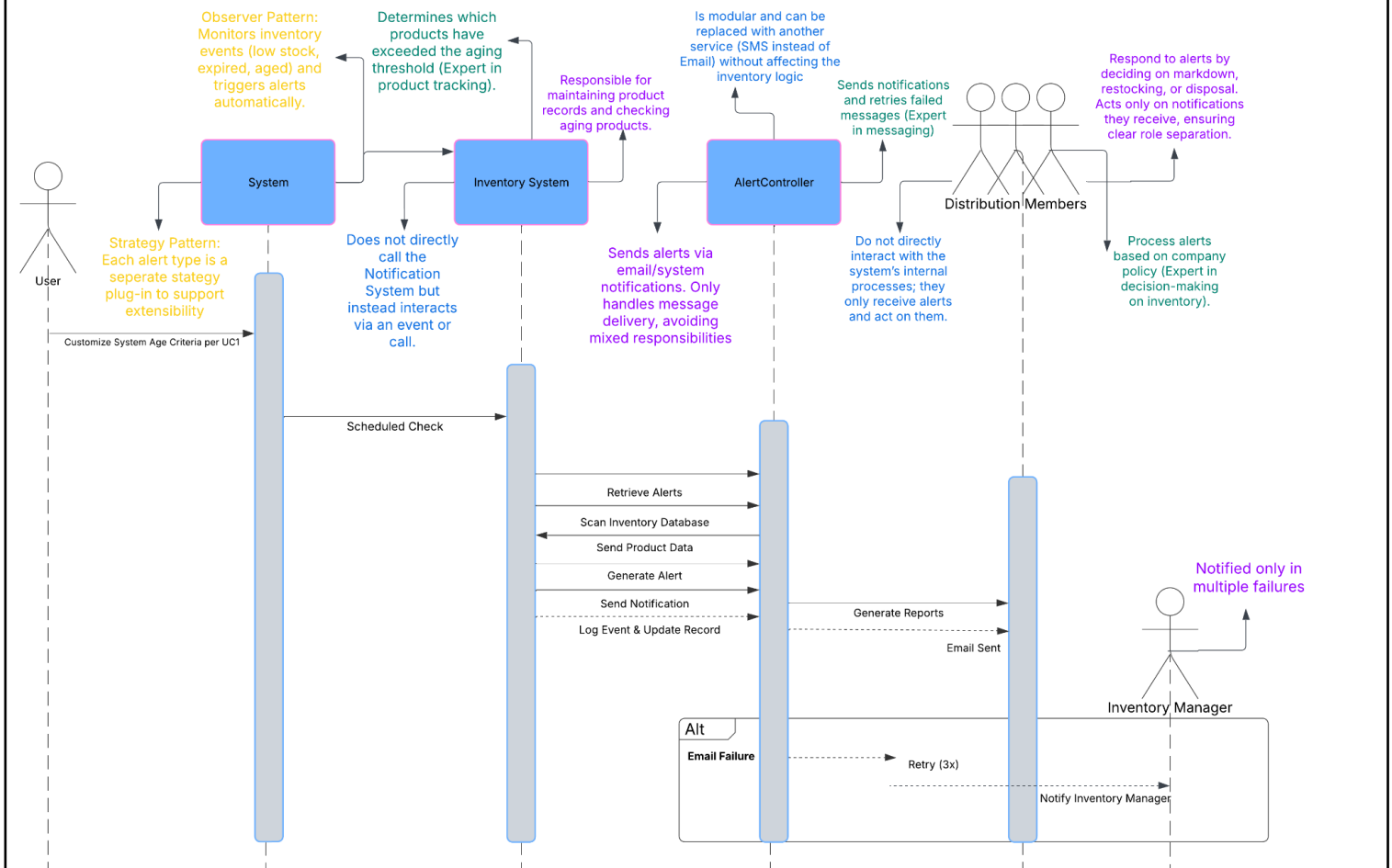
Use Case UC-1: Database Customization



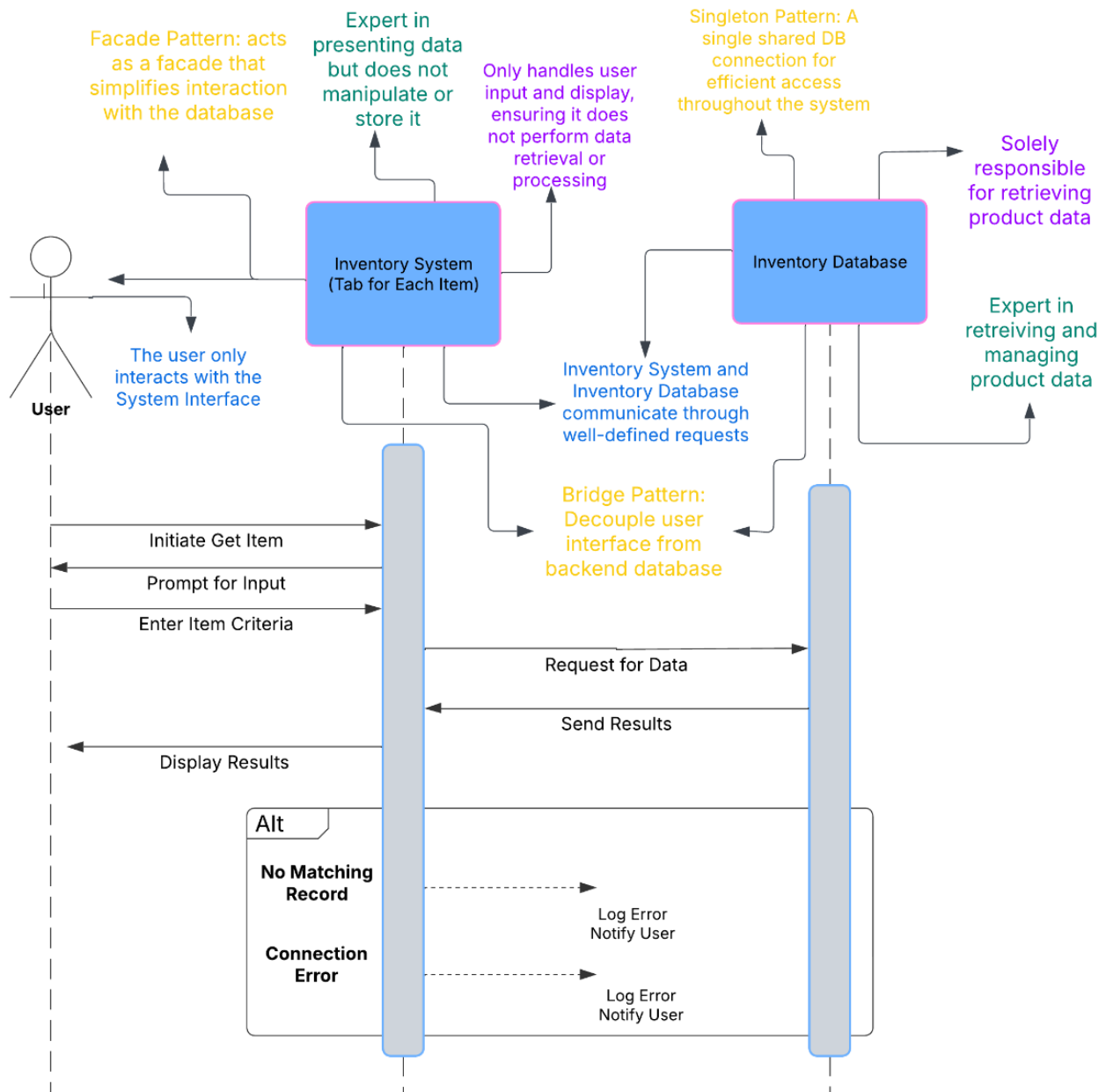
Use Case UC-2: Product to Vendor/Sales Linking



Use Case UC-4: Provide Alert Notifications for Aging Products



Use Case UC-5: Searching the Database

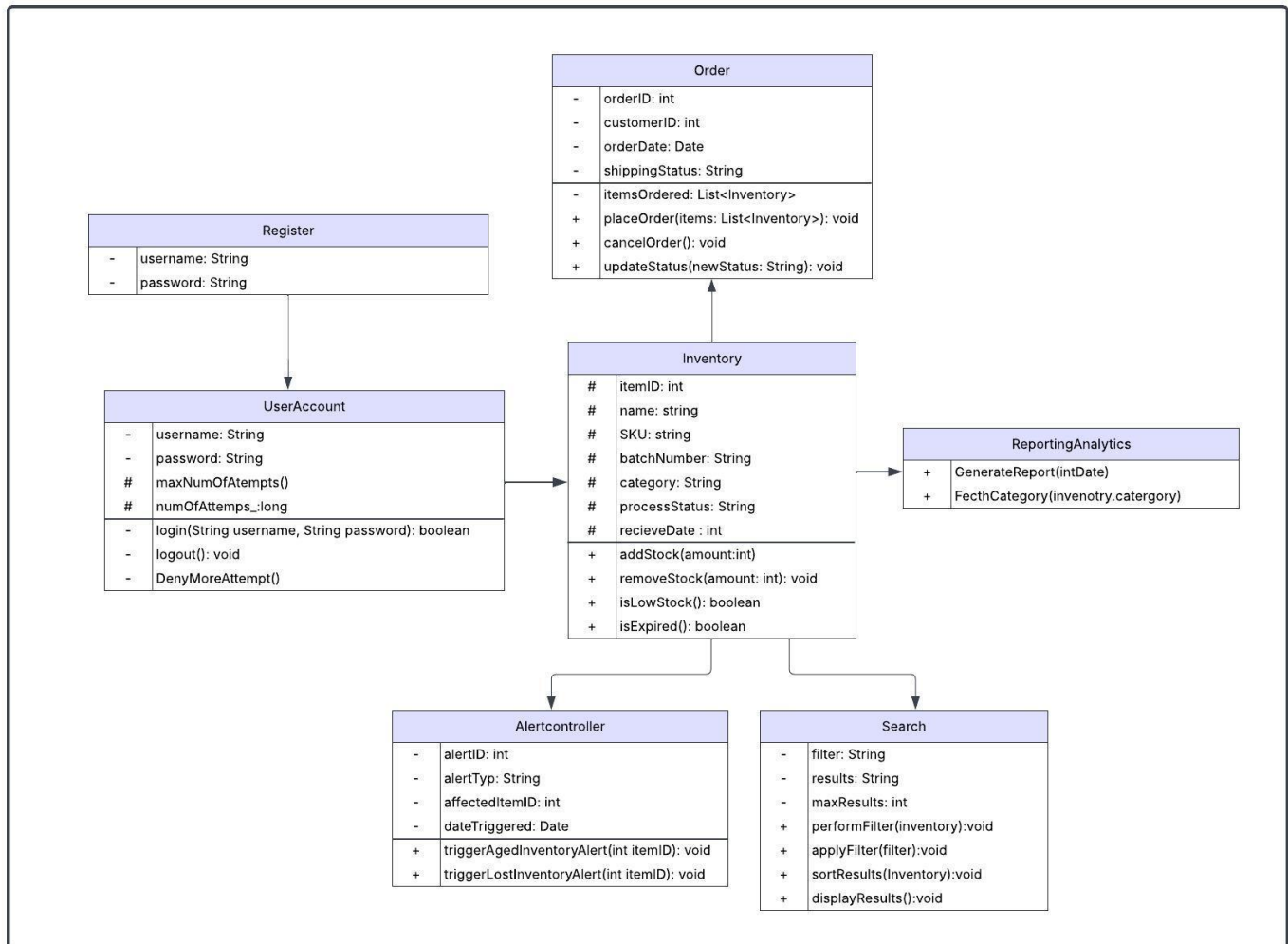


UC Interaction Diagrams Link:

https://lucid.app/lucidchart/04d00800-7baa-4f2f-86f2-18a9a6fc2d89/edit?viewport_loc=-127%2C3078%2C3211%2C1435%2C0_0&invitationId=inv_4939c98d-0946-445d-b50c-2228af115a56

Class Diagram and Interface Specification

Class Diagram



Link: https://lucid.app/lucidchart/313460b2-85ba-4584-a1eb-4ac560b7a97d/edit?page=0_0&invitationId=inv_7a3c4bb2-db12-41e6-a661-ec808aed77da#

Data Types and Operation Signatures

This section breaks down the key parts of the system, explaining what each class does, what kind of data it holds, and the actions it can perform. This helps ensure everything in the system is clear and functions smoothly.

1. Inventory Class

This class keeps track of everything in stock. It records details like item names, how much is available, and when items expire.

- **Attributes:**

- itemID: int - Unique number assigned to each inventory item.
- name: String - Name of the product.
- SKU: String - Stock-keeping unit (SKU) used for tracking.
- batchNumber: String - Identifies which batch the item is from.
- category: String - Describes what type of item it is (e.g., perishable, electronics, etc.).
- processedStatus: String - Current status of the item (e.g., received, allocated, shipped).
- receivedDate: Date - The date when the item was added to inventory.
- expirationDate: Date - If applicable, the date when the item expires.

- **Operations:**

- addStock(amount: int): void - Adds more of an item to inventory.
- removeStock(amount: int): void - Reduces the stock when an item is sold or removed.
- isLowStock(): boolean - Checks if there's not much left of an item.
- isExpired(): boolean - Determines if an item has expired.

2. Order Class

This class handles everything related to orders, from placing them to tracking their status.

- **Attributes:**

- orderID: int - Unique number assigned to each order.
- customerID: int - Links the order to the customer who placed it.
- orderDate: Date - The date when the order was placed.
- shippingStatus: String - Status of the order (e.g., pending, shipped, delivered).
- itemsOrdered: List<Inventory> - List of items included in the order.

- **Operations:**

- placeOrder(items: List<Inventory>): void - Creates a new order and updates inventory.
- cancelOrder(): void - Cancels an order before it ships.
- updateStatus(newStatus: String): void - Changes the order's status as it moves through processing.

3. AlertController Class

This class is in charge of sending notifications when important inventory events happen, like when stock is low or items are missing.

- **Attributes:**
 - alertID: int - Unique number assigned to each alert.
 - alertType: String - Type of alert (e.g., low stock, expiring item).
 - affectedItemID: int - The inventory item the alert is about.
 - dateTriggered: Date - The date when the alert was sent.
- **Operations:**
 - triggerAgedInventoryAlert(int itemID): void - Sends an alert for items close to expiring.
 - triggerLostInventoryAlert(int itemID): void - Notifies users when an item is missing from inventory.

4. UserAccount Class

This class manages user login and authentication, making sure only authorized users can access the system.

- **Attributes:**
 - username: String - The user's login name.
 - password: String - The user's encrypted password for security.
- **Operations:**
 - login(String username, String password): boolean - Checks if login details are correct and grants access.
 - logout(): void - Logs the user out of the system.

This section ensures all major parts of the system are defined and function as expected. Each class plays a key role in keeping inventory organized, processing orders, managing alerts, and securing user access.

5. ReportingAnalytics Class

This class is in charge of generating reports and fetching product data. This class will be used to create visualizations without directly interacting with the inventory systems

- **Operations**
 - generateReport(inventory) - generates a report on the inventory information
 - fetchCategory(inventory.category) - allows you to specify specific categories to pull information from those categories.

6. Search class

This class is in charge of taking in users input and searching inventory depending on the needs of the users. The Search class will search through the database to find matches once the results are found and organized, it displays them in a user-friendly way.

- **Attributes**

- filter: String - This attribute will be used to refine the search
- results: String - This attribute will be used to store the results and return the results.
- maxResults - This attribute will be used to manage the number of results showing up to the user.

- **Operations**

- `performFilter(inventory)void`: This operation will execute the search using the query and data source.
- `applyFilter(filter)void`- This operation will Refine search results based on criteria.
- `sortResults(inventory)void`- This operation will sort the results by relevance filter the user input.
- `displayResults()void`: This operation will output the formatted search results.

Traceability Matrix

[illegible]

registerController

- Account Access: Enables the user to register for an account.

userAccount

- Account Access: Stores the users login information.

authController

- Account Access: Authenticates the user to login to the database.

logoutController

- Account Access: Logs the user out of the database.

rolesController

- Account Access: Limits users access to the database.

Inventory

- Inventory Data: Displays all the information within the inventory table.
- Orders: Connects all order information to the inventory.
- Search: Enables the user to search through inventory information.

companyDivisionController

- Inventory Data: Displays the company divisions and connects them to the inventory table.

locationController

- Inventory Data: Displays the location of items within the inventory.

customerController

- Inventory Data: Displays all customer information for all orders placed against the inventory.

orderController

- Inventory Data: Displays all orders that customers have made from available inventory.

vendorController

- Inventory Data: Displays all vendor information in a type of profile for specific inventory owned by the customer.

shippingController

- Inventory Data: Displays all shipping information for orders placed by customers from the inventory.

agingInventoryController

- Inventory Data: Displays aging inventory from the current inventory.
- Alerts: Displays all information for alerts.

alertController

- Alerts: Tracks the low stock, expired, and aged items.

alertEmailJob

- Alerts: represents the notification created to send to a list of recipients via email everyday

Design Patterns

Our project has evolved using multiple different design patterns across the entire system. The high-level architecture we have implemented using the Model-View-Controller (MVC) has helped to keep responsibilities separate using models, routes, and controllers, which keep the code easier to test and maintain.

Some of the system wide design patterns we have implemented are listed below:

1. Observer Pattern - Alert System Behavior

This portion of the system “observes” different item conditions (low stock, expired, aged) and triggers alerts.

2. Singleton Pattern - Database Pool

This shared instance pool from the db.js is used throughout the app, this ensures there is only one DB connection manager for the system, improving performance and resource control.

3. Strategy Pattern - Different Alert Check Types

Our system has different strategies (low stock etc) for generating alerts implemented as separate functions (checkLowStockAlerts, etc) that plug into a shared alert interface. This supports the system in being easily extendable to add additional strategies.

4. Command Pattern - Trigger Jobs

runAllAlerts acts as a command that encapsulates a series of operations, making it easy to reuse and manually run via API or during testing.

5. Facade / Mediator - API / GUI

Our system uses the API and user interface acting as a Facade and Mediator that simplifies interactions for the user and the database.

6. Environment Configurations - .env

Our system uses environment configuration files to securely manage sensitive information, such as user roles and login credentials. This approach enhances system security and helps prevent the accidental exposure of confidential data.

Object Constraint Language (OCL) Contracts

Login in screen

- **Context:** UserAccountController::login(inputUsername: String, inputPassword: String)
- **Invariant:** not self.username.empty() and not self.password.empty()
- **Precondition:** inputUsername = self.username and inputPassword = self.password
- **Postcondition:** result = true

- **Context:** RegisterController::registerUser(username: String, password: String)
- **Invariant:** UserAccount.allInstances()->exists(u | u.username <> "")
- **Precondition:** UserAccount.allInstances()->forall(u | u.username <> username)
- **Postcondition:** UserAccount.allInstances()->exists(u | u.username = username and u.password = password)

API

- **Context:** Inventory::addItem(), Inventory::modifyItem(), Inventory::deleteItem()
- **Invariant:** UserAccount.allInstances()->exists(u | u.isAdmin = true)
- **Precondition:** UserAccount.allInstances()->exists(u | u.isAdmin = true)
- **Postcondition:** Inventory.allInstances()->exists(i | i.itemID = itemID, i.name = name, i.SKU = SKU, i.batchNumber = batchNumber, i.processStatus = processStatus, i.receiveStock = receiveStock)

- **Context:** Inventory::getItem()
- **Invariant:** UserAccount.allInstances()->exists(u | u.isUser = true)
- **Precondition:** Inventory.allInstances()->exists(i | i.itemID = itemID)
- **Postcondition:** post: result <> null

- **Context:** Order::getShippingInfo(orderID: String)
- **Invariant:** vendor.getShippingInfo(orderID) = Shipper.allInstances()->select(s | s.hasOrder(orderID))
- **Precondition:** Order.allInstances()->exists(v | v.hasPlacedOrder(orderID))
- **Postcondition:** result <> null

Alerts

- **Context:** AlertController::generateAlert(alertID: String, alterType: String, affectedItem: int, dateTrigger: Date)
- **Invariant:** UserAccount.allInstances()->forall(u | u.receivesAlerts = true)

- **Precondition:** `Inventory.allInstances()->exists(i | i.itemID = itemID and (i.quantity < i.setQuantity or i.expired = true or i.ageInDays > i.thresHoldDays or i.demand < 10))`
- **Postcondition:** `result = true`
- **Context:** `AlertEmailJob::sendEmailAlert(itemID: Int)`
- **Invariant:** `UserAccount.allInstances()->forall(u | u.receivesEmailAlerts = true)`
- **Precondition:** `Inventory.allInstances()->exists(i | i.itemID = itemID and (i.quantity < i.setQuantity or i.expired = true or i.ageInDays > i.thresHoldDays or i.demand < 10))`
- **Postcondition:** `result = true`

Algorithms and Data Structures

Algorithms

Fuzzy Search Algorithm

Uses Levenshtein Distance to compare the search input with stored inventory names and return the closest matches.

Formula:

Levenshtein Distance calculates how many changes (insertions, deletions, or substitutions) are needed to match two words.

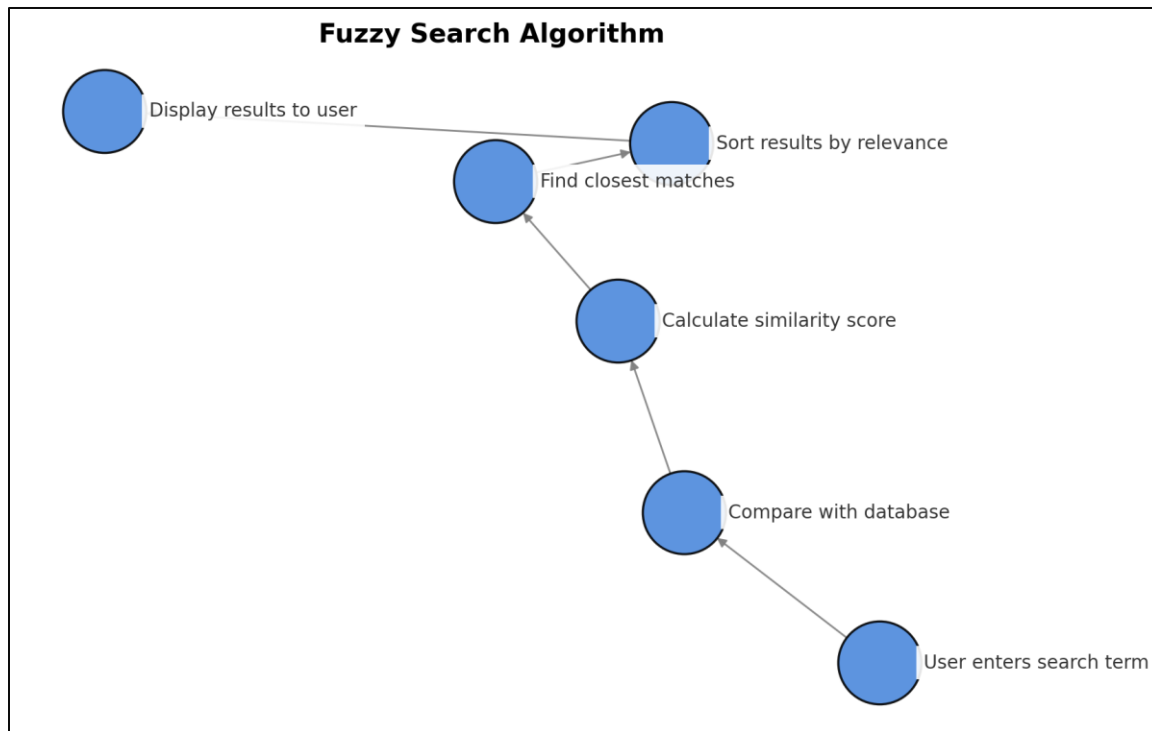
$$D(i,j) = \min \{ D(i-1, j) + 1, D(i, j-1) + 1, D(i-1, j-1) + \text{cost}(i,j) \}$$

Where:

- $D(i,j)$ is the edit distance between two strings.
- $\text{cost}(i, j) = 0$ if characters match, 1 otherwise.

Implementation:

- Compares search input with inventory items.
- Finds and ranks the closest matches.
- Displays the best results to the user.



Economic Order Quantity (EOQ) Algorithm

Determines the optimal reorder quantity to minimize inventory costs and balance ordering vs. holding costs.

Formula:

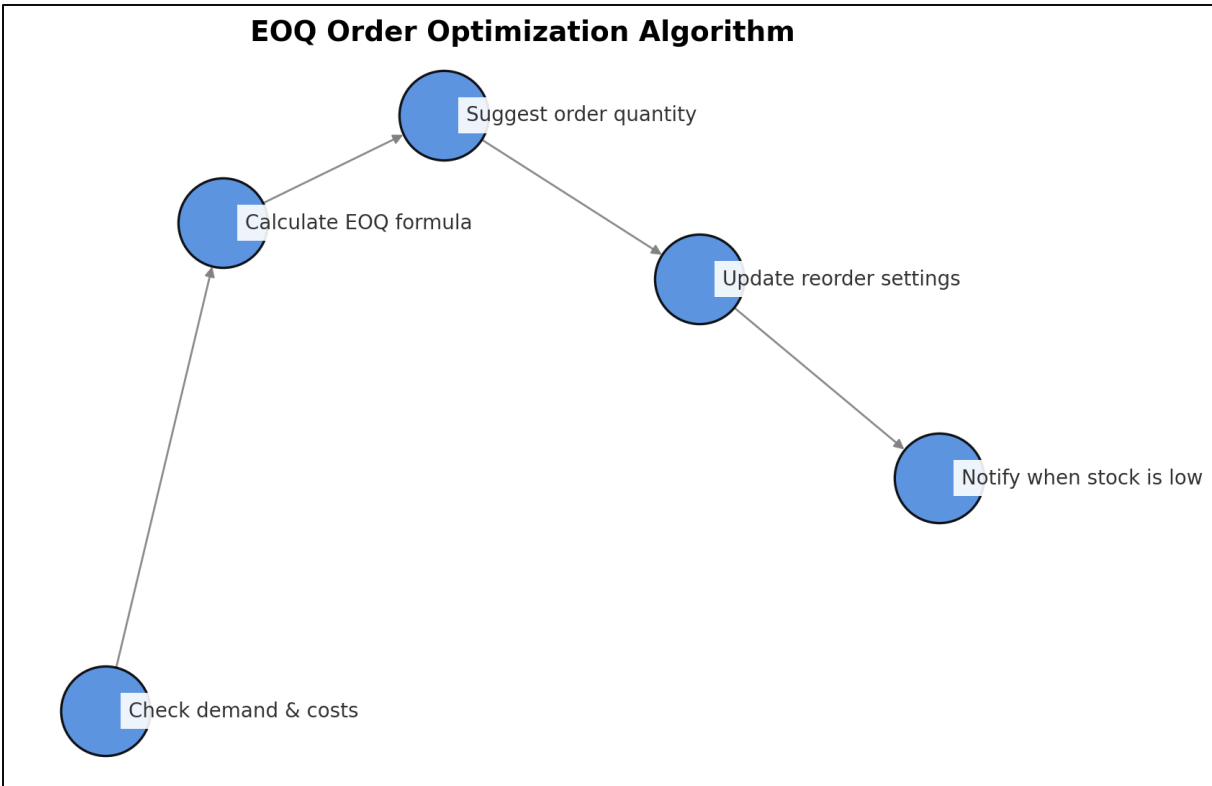
$$EOQ = \sqrt{(2DS) / H}$$

Where:

- D = Demand (units per year)
- S = Ordering cost per order
- H = Holding cost per unit per year

Implementation:

- The system calculates EOQ automatically.
- Suggests the best reorder quantity when stock is low.



Stock Monitoring Algorithm

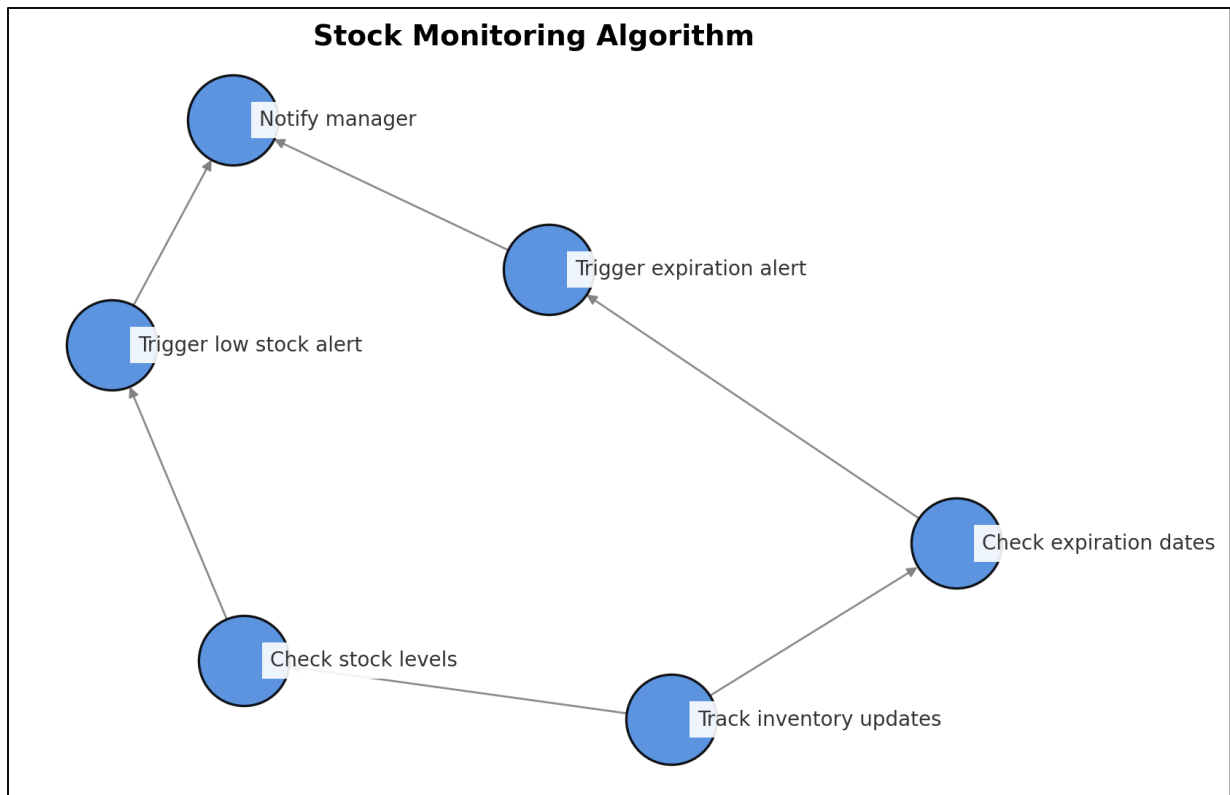
Tracks stock levels and expiration dates, triggering alerts for low stock and expired items.

Mathematical Conditions:

- Low Stock Alert: $\text{Alert} = \text{true}$ if $\text{currentStock} < \text{threshold}$
- Expiration Alert: $\text{Alert} = \text{true}$ if $\text{expirationDate} \leq \text{currentDate}$

Implementation:

- Checks if stock is below the reorder threshold.
- Flags expired items.
- Sends alerts for low stock or expired products.
- Notifies the manager for restocking or removal.



Order Processing Algorithm

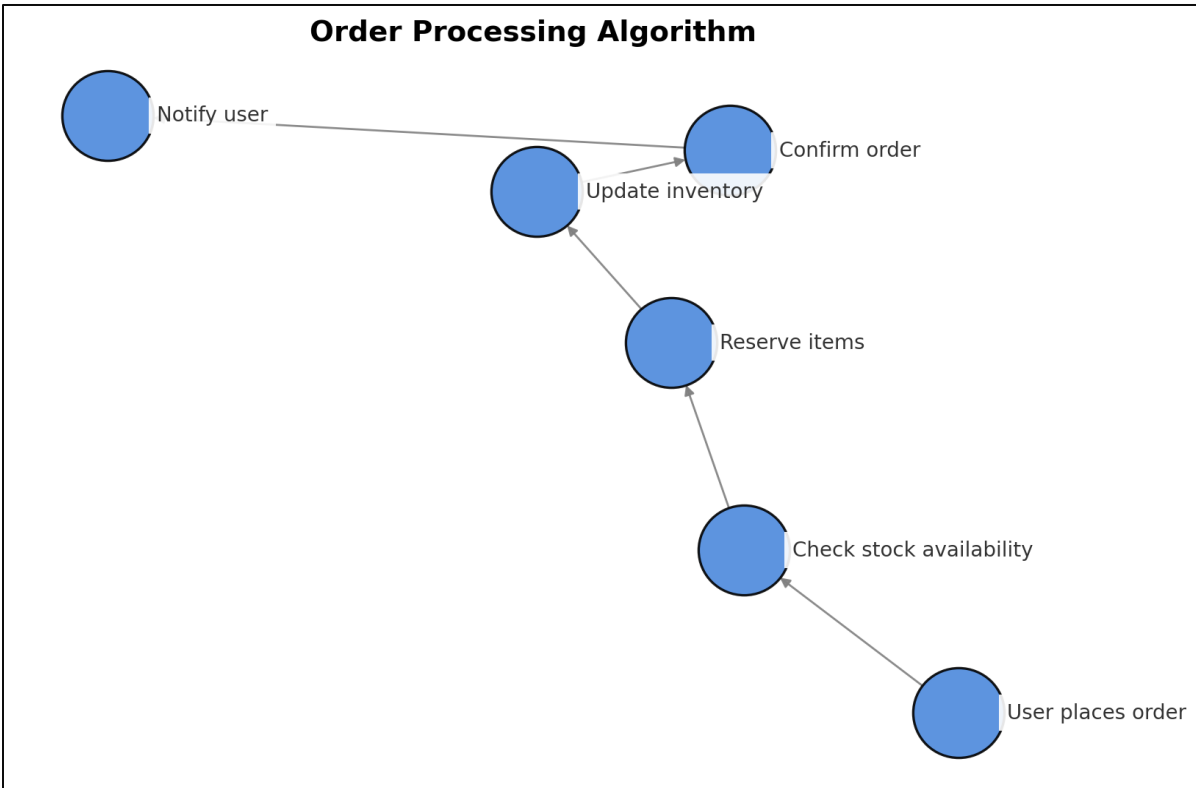
Ensures orders are only processed if there is enough stock and prevents overselling.

Mathematical Conditions:

- Stock Check Before Order Processing: $\text{sum}(\text{stock}_i) \geq \text{sum}(\text{ordered}_i)$
- Stock Update After Order Processing: $\text{newStock}_i = \text{stock}_i - \text{ordered}_i$

Implementation:

- Checks if enough stock exists before confirming an order.
- Reserves stock and updates inventory upon order placement.
- If stock is too low, place the order on hold and alerts the manager.



Reporting & Analytics Algorithm

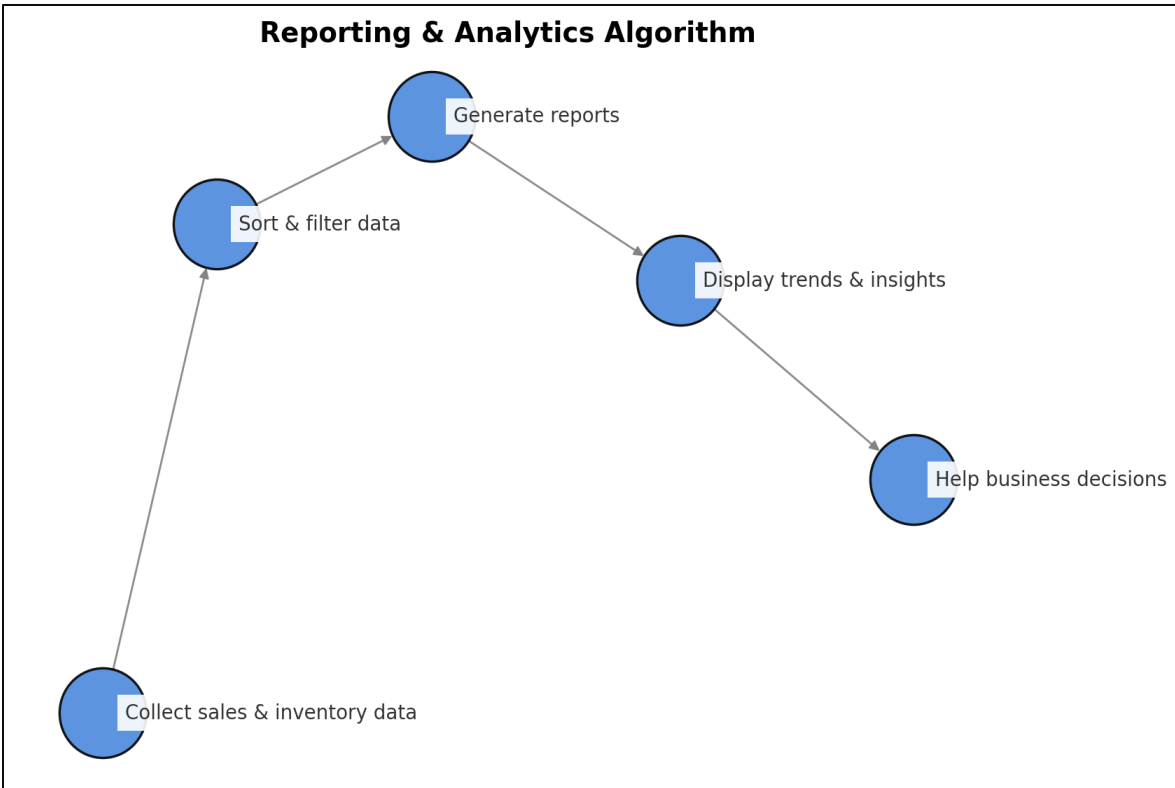
Generates reports on inventory levels, stock movement, and sales trends.

Mathematical Conditions:

- Filter by Category: $\text{FilteredInventory} = \{ i \mid i \in \text{Inventory}, i.\text{category} = \text{selectedCategory} \}$
- Sort by Date: $\text{SortedInventory} = \text{sort}(\text{FilteredInventory}, \text{key} = \text{receivedDate})$

Implementation:

- Collects and organizes inventory data.
- Sorts data by category and date.
- Displays reports with insights on stock trends.



Summary

These algorithms ensure efficient inventory management by reducing waste, preventing stockouts, and improving order processing. They use mathematical models to optimize stock levels and business decisions.

Data Structures

Our system runs on multiple devices and requires smooth communication between the frontend, backend (Node.js), and database (PostgreSQL). To ensure efficiency and scalability, we use the following data structures:

1. Inventory Data Structure

- Data Structure Used: JavaScript Object (JSON) / PostgreSQL Database
- Why? JavaScript objects allow fast lookups for inventory data, and PostgreSQL ensures data is stored permanently using B-Tree indexes for efficient searches.

- **How It Works:**

- o The backend (Node.js) retrieves inventory using SQL queries via the PostgreSQL driver (pg).

- o The frontend requests inventory data through a REST API (HTTP/HTTPS).
- o The backend processes the request and returns the response in JSON format.

2. Order Management Data Structure

- Data Structure Used: Queue (JavaScript Array) / PostgreSQL Table
- Why? Orders need to be processed in the order they were placed (FIFO - First-In-First-Out). PostgreSQL stores pending orders and retrieves them sequentially.

- **How It Works:**

- o A user places an order on the frontend, sending an HTTP POST request to the backend.
- o The backend (Node.js) checks stock in PostgreSQL using SQL queries.
- o If stock is available, the order is added to the processing queue.

3. Search & Filtering Data Structure

- Data Structure Used: JavaScript Map / PostgreSQL B-Tree Index
- Why? JavaScript Maps allow fast lookups, while PostgreSQL B-Trees make database searches more efficient by indexing key columns.

- **How It Works:**

- o The frontend sends a search request (e.g., GET `/search?query=apple``).
- o The backend (Node.js) first checks the Map (if cached), then queries PostgreSQL for more results.
- o The response is returned in JSON format via the REST API.

4. User Accounts & Authentication Data Structure

- Data Structure Used: JavaScript Object / Secure PostgreSQL Table
- Why? Ensures fast login verification and secure password storage using hashed passwords.

- **How It Works:**

- o Users log in through the frontend, sending an HTTPS request with credentials.

- o The backend (Node.js) hashes the password and checks the PostgreSQL database.
- o If correct, the user is authenticated and issued a JWT token for secure access.

Why We Chose These Data Structures

We selected these based on performance, scalability, and integration with Node.js and PostgreSQL:

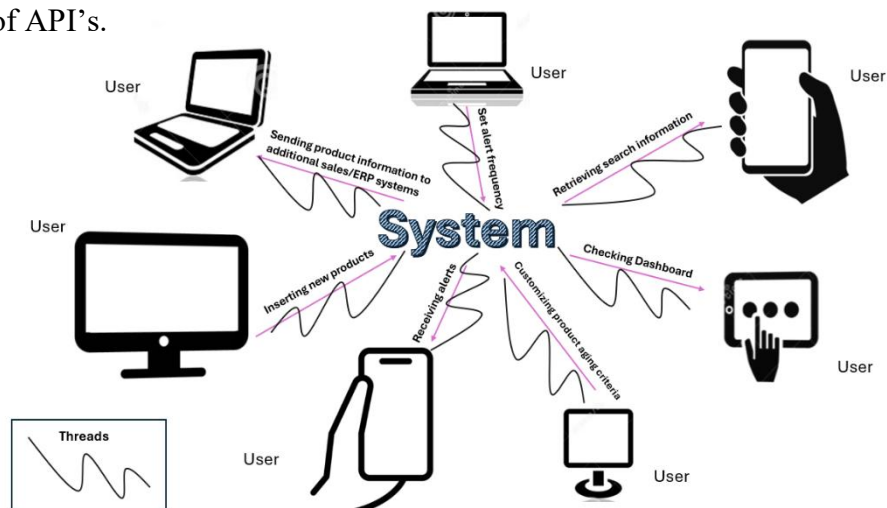
- PostgreSQL uses B-Trees for indexing, making searches faster.
- JavaScript Maps and Objects provide fast in-memory lookups for frequently used data.
- Queues (Arrays) allow smooth order processing in real-time.
- REST APIs ensure seamless communication between the frontend and backend.

Summary

Our system uses JavaScript Objects, Maps, Queues (Arrays), and PostgreSQL B-Trees to efficiently handle inventory tracking, order processing, user authentication, and search functions. These data structures work with Node.js, PostgreSQL, and REST APIs to ensure fast and reliable communication across devices.

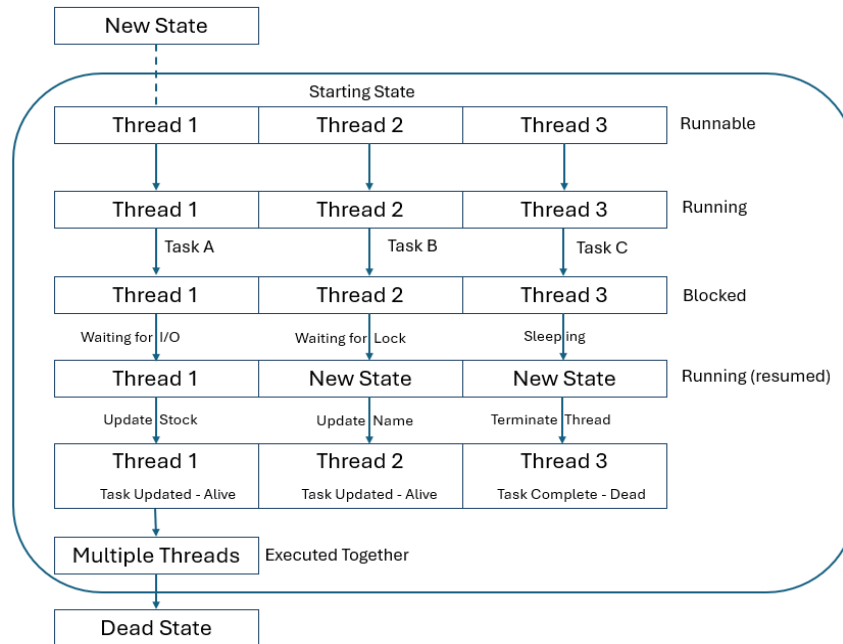
Concurrency: Does your system use multiple threads?

“Threading is a segment which divide the code into small parts that are of very light weight and has less burden on CPU memory so that it can be easily worked out and can achieve goal in desired field (GeeksforGeeks, 2024).” Our product requires multiple threads to ensure a total product useability for the customer. From the image below, you are able to see the threads of the system as direct inputs and outputs of the system. Without multiple threading and multitasking, our system would not stand up to ease of use criteria outlined in our customer requirements. Multiple threads allow for our system to be adaptable and user friendly over a wide range of API's.



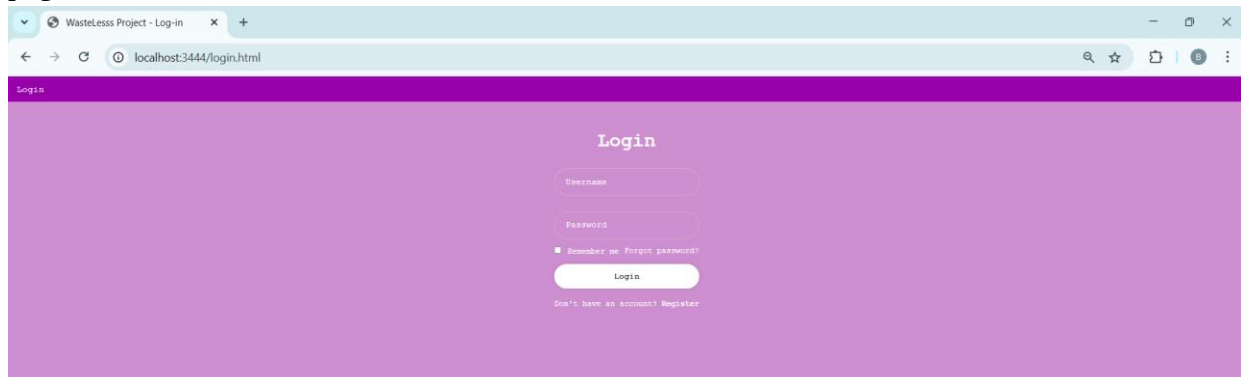
In order for the system to be adaptable across platforms, we will use concurrency by using object controllers (e.g. *InventoryController*, *AlertController*, etc). This method will allow for the system to juggle two distinct tasks. To ensure that the two distinct tasks can run in parallel, our system uses a controller thread, implemented CRUD in a thread safe manner by using “atomic ID updates” Yahia (2025), using Promise.all() with async/await to simulate concurrent requests and (ConcurrentQueue) to manage products.

Multi-Thread State Diagram

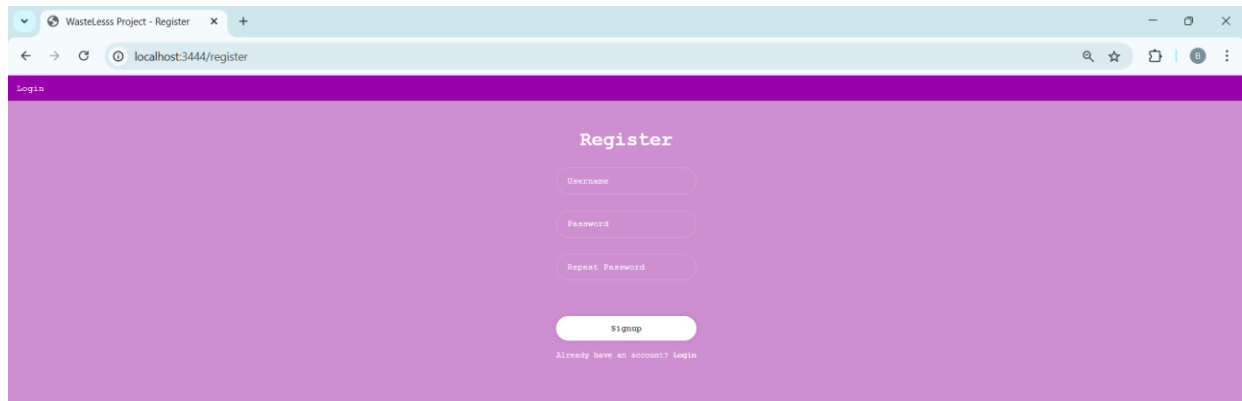


User Interface Design and Implementation

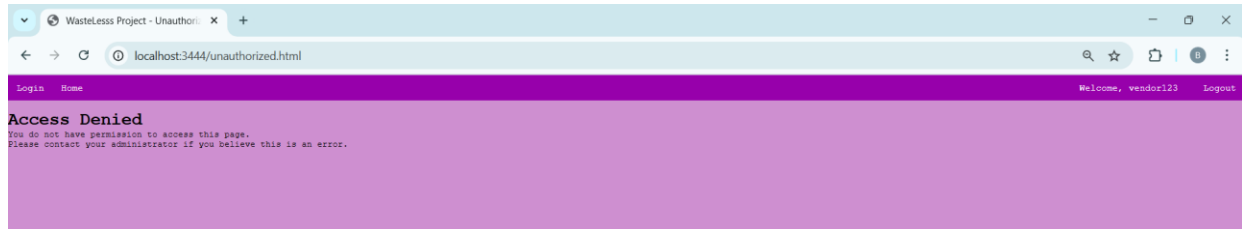
Screenshot below shows the log-in page where the user will enter their username and password. If the user does not have log-in information they are directed to click on the register button. This page relates back to Use Case 1.



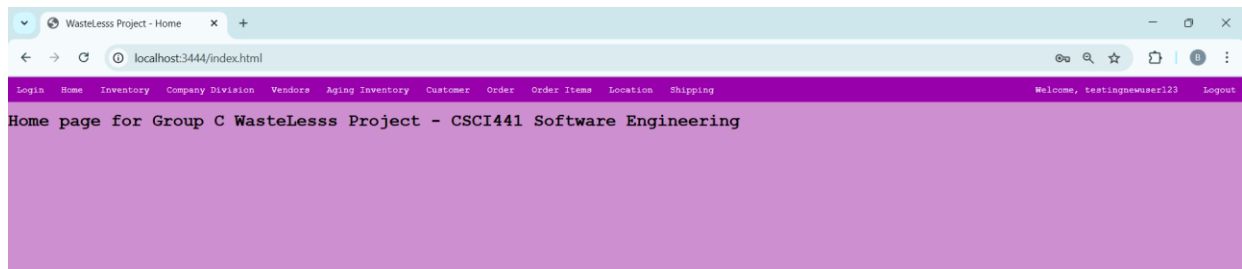
Screenshot below shows the register page where the user will enter a username, password, and repeat the password before clicking the signup button. This page relates back to Use Case 1.



Screenshot below show the unauthorized user page where the user does not have access to the page they attempted to access. This page relates back to Use Case 1.



Screenshot below shows the home page where the user will find themselves after logging in. From here the user can use the navigation bar at the top to access any pages they have access to. This screenshot relates back to all Use Cases.



Screenshot below shows the inventory page where the user can will receive inventory alerts and create, get, update, and delete information in the inventory table. This page relates back to Use Case 1 and 4.

WasteLess Project - Inventory

localhost:3444/inventory.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Inventory

Get Alerts

Clear All Alerts

Manage Inventory

Item ID (for update only):

Name:

SKU:

Batch Number:

Category:

Processed Status:

Received Date: mm/dd/yyyy

Expiration Date (optional): mm/dd/yyyy

Location ID:

Is Perishable: ☐

Shelf-life Days:

Screenshot below shows the company division page where the user can create, get, update, and delete information in the company division table. This page relates back to Use Case 2.

WasteLess Project - Company

localhost:3444/companyDivision.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Company Division

Company Divisions

Division ID (for update only):

Division Name:

Manager:

Add Company Division

Update Company Division

Delete Company Division

Division ID (for delete only):

Delete Company Division

Screenshot below shows the vendors page where the user can create, get, update, and delete information in the vendors table. This page relates back to Use Case 2.

The screenshot shows a web browser window with the URL `localhost:3444/vendor.html`. The page has a purple header with a navigation menu: `Login`, `Home`, `Inventory`, `Company Division`, `Vendors`, `Aging Inventory`, `Customer`, `Order`, `Order Items`, `Location`, `Shipping`. The user is logged in as `testingnewuser123`. The main content area is purple and contains the following sections:

- Get Vendors**: A button to retrieve vendor information.
- Manage Vendors**: A section for managing vendors with input fields for:
 - Vendor ID (for update only):
 - Vendor Name:
 - Contract Info:
 - Address:
- Add Vendor**: A button to add a new vendor.
- Update Vendor**: A button to update an existing vendor.
- Delete Vendor**: A section for deleting a vendor with an input field for:
 - Vendor ID (for delete only):
- Delete Vendor**: A button to delete a vendor.

Screenshot below shows the aging inventory page where the user can create, get, update, and delete information in the aging inventory table. This page relates back to Use Case 3.

The screenshot shows a web browser window with the URL `localhost:3444/agingInventory.html`. The page has a purple header with the same navigation menu as the Vendors page. The user is logged in as `testingnewuser123`. The main content area is purple and contains the following sections:

- Get Aging Inventory**: A button to retrieve aging inventory information.
- Company Divisions**: A section for managing aging inventory with input fields for:
 - Aging ID (for update only):
 - Item ID:
 - Aging Date:
 - Quantity:
- Add Aging Inventory**: A button to add a new aging inventory item.
- Update Aging Inventory**: A button to update an existing aging inventory item.
- Delete Aging Inventory**: A section for deleting an aging inventory item with an input field for:
 - Aging ID (for delete only):
- Delete Aging Inventory**: A button to delete an aging inventory item.

Screenshot below shows the customer page where the user can create, get, update, and delete information in the customer table. This page relates back to Use Case 2.

WasteLess Project - Customer

localhost:3444/customer.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Customer

Customer

Customer ID (for update only):

Customer Name:

Email:

Phone:

Address:

Add Customer

Update Customer

Delete Customer

Customer ID (for delete only):

Delete Customer

Screenshot below shows the orders page where the user can create, get, update, and delete information in the orders table. This page relates back to Use Case 2.

WasteLess Project - Order

localhost:3444/order.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Order

Orders

Order ID (for update only):

Customer ID:

Order Date:

Shipping Status:

Add Order

Update Order

Delete Order

Order ID (for delete only):

Delete Order

Screenshot below shows the order items page where the user can create, get, update, and delete information in the order items table. This page relates back to Use Case 2.

WasteLess Project - Order Item

localhost:3444/orderItems.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Order Item

Order Items

Order Item ID (for update only):

Order ID:

Item ID:

Quantity:

Add Order Item

Update Order Item

Delete Order Item

Order Item ID (for delete only):

Delete Order Item

Screenshot below shows the location page where the user can create, get, update, and delete information in the location table. This page relates back to Use Case 2 and 5.

WasteLess Project - Location

localhost:3444/location.html

Login Home Inventory Company Division Vendors Aging Inventory Customer Order Order Items Location Shipping Welcome, testingnewuser123 Logout

Get Location

Locations

Location ID (for update only):

Customer ID:

Address:

Capacity:

Add Location

Update Location

Delete Location

Location ID (for delete only):

Delete Location

Screenshot below shows the shipping page where the user can create, get, update, and delete information in the shipping table. This page relates back to Use Cases 2 and 5.

Test Designs

Test-case identifier: TC-1 Use Case Tested: UC- 1 Pass/Fail Criteria: The test will pass when the user is actively able to add products, pricing, and age criteria into the database. If the database does not successfully add the item (duplicate item) a notification displays	
Test Procedure	Expected Result:
Step 1. Navigate to the directed API link	When users click on the link they are automatically directed to the login screen - this will ensure the system API is accessible and provides the ability for the user to login to the system.
Step 2. Input login information	User is able to submit login information and the system takes you to the visual dashboard for your system.
Step 3: Navigate to the area for adding new items into the system.	Users are able to access a specific area for adding new items. When the user goes to the area for customizing items, the GUI allows for user input. When criteria is inputted the system accurately submits the POST and saves the appropriate data in the correct location

Step 4: Add new item in the input areas (Data: Product Name, Age Criteria, Vendor Name) Step 5: Require Data Inputs for Demand / Ordering cost per order / Holding cost per unit per year Step 5: Add the same item again (test for system error)	in the database. The data auto implements an incremented SKU ID with DATE anytime a new 'PRODUCT' is entered. User input provides categorization and notifications based on algorithms for EOQ, Stock Monitoring, and Order Processing. The system should display an error message that the exact product already exists
--	---

Test-case identifier: TC-2

Use Case Tested: UC- 5

Pass/Fail Criteria: The test will pass when the user is actively able to search for items that are already added into the database. If the item does not exist, an error message will display.

Test Procedure	Expected Result:
Step 1: Navigate to the area for accessing the Search Tool Step 2: Input item data (SKU, product name, age timeframe) for an item previously adding in the database Step 3: Input item data (SKU, product name, age timeframe) for an item NOT previously adding in the database	Users are able to access a specific area for adding searching items. When the user goes to the area for customizing items, the GUI allows for user input. When criteria is inputted the system accurately submits the GET as requested in the API, and displays the appropriate requested result for the user. The system should display an error message that the product doesn't exit

Test-case identifier: TC-3

Use Case Tested: UC- 2

Pass/Fail Criteria: The full test will pass when the user is actively able to link an already established product, and an

Test Procedure	Expected Result:
Step 1: Navigate to a product previously added into the database.	The system should display the appropriate requested result for the user. In this case we should see the vendor product information and if a vendor is already linked.

Step 2: Add the vendorId of the vendor previously added into the database and press UPDATE	The system then re-routes you back to the customization area for the system to ADD characteristic to current database item.
Step 3: Add known vendorID to pre-existing item.	System successfully links the product to the already established vendor.
Step 4: Add unknown vendorID to pre-existing item.	System displays error message that the vendor does not exist

Test-case identifier: TC-4 Use Case Tested: UC- 3 Pass/Fail Criteria: The test will pass if the user can retrieve information from the API with their specified credentials. The test will fail if the inventory information is displayed without detailed information of the aged inventory. Input Data: SKU Number, Age Criteria	
Test Procedure	Expected Result:
Step 1. The user incorrectly types in the SKU number to retrieve information.	The System displays message indication inventory was not found and prompts the user to try again.
Step 2. The user correctly types in the SKU.	The system successfully retrieves records located in the database matching the SKU number with the specified criteria categorized by age.

Test-case identifier: TC - 5 Use Case Tested: UC - 4 Pass/Fail Criteria: Test will pass if the user receives notification of inventory that has been categorized by age and is close to the expiration date when correct input is provided. Test will fail if the user does not receive notification of inventory that has been categorized by age and is close to the expiration date. Input Data: One or more email addresses, or approve of system notifications	
Test Procedure	Expected Result:
Step 1: The user navigates to the Inventory tab	The system changes from the home screen to the inventory tab screen
Step 2. The user interface asks the user if they want to enable notifications.	The system displays a message asking the user if they are sure they do not want to turn on system notifications.

Step 3. The user allows system notifications and the user provides a valid email address for the system admin.	The system displays a message thanking the user for turning on system notifications. Then the user interface displays notifications for specific alerts based on inventory criteria.
--	--

Testing Coverage

Our unit testing is completed in parallel with specific classes or modules during development. The testing will ensure that different modules or components of software work seamlessly together. Since inventory systems often interact with other systems, we will verify unit success before moving on to the module to ensure seamless integration between the two units. Our integration testing is outlined as user acceptance test cases above. For instance, in developing the back-end API linkage we will ensure each individual file links and displays the correct information before integrating the API with the rest of the system. Our test coverage will focus on equivalence testing and boundary testing to narrow down the type of code coverage we will do. The system will test specific modules that retrieve inventory by dividing input data into equivalent groups, then we will test one representative value from each group to test that the program behaves the same on each group. This allows us to make the assumption that the system will behave similarly when validating different input categories, such as item IDs, inventory quantities, or pricing tiers. Our test coverage will also incorporate boundary testing which will focus on special cases that have parameters. This will be crucial to testing the reliability of the software because the system will need to rely on strict limits for stock levels, reordering, and alert notification for aging software. For example, when applying equivalence testing to aged inventory will input data into equivalent groups such as the number of days the inventory has been in stock such as 0 - 30 days, 30 - 90 days, and 91+ days. Then in boundary testing we will focus on the edge of these equivalence groups to ensure the system handles the transition between each group as expected.

Integration Testing

Our integration testing strategy will start by identifying the different integration points of our system. The components that we will integrate, and test will be our database, CRUD API, email API, inventory management module, and web dashboard. The integration testing strategy we will use is the bottom-up integration approach where we will start by combining the units at the lowest level of hierarchy. We will build our lower-level units without dependencies and work upwards. For example, we will start by building our database first and use the boundary testing to execute postgresql statements to test the cohesiveness of our entities.

The integration will proceed with the CRUD API, and email API. As we start proceeding to the higher-level unit's integration of the modules, we will build a test driver to test the response of the lower-level units. Usually in a bottom-up integration there is no need to develop test stubs but in this case since our dashboard for our web interface has not been built yet we will need to simulate interactions with the APIs through test stubs to ensure the reliability of the system. By following the bottom-up approach with our integration testing the system will have a solid foundation of each component by ensuring that low level components are working as intended before integration. Having a solid foundation will help minimize cascading errors and ensure a smooth integration process.

Outline of Algorithm and System Specific Tests

Back-End Testing:

- *Database Testing:*
 - Verify successful database creation and schema integrity
 - Test database connectivity with the API
 - Validate consistency and integrity after CRUD operations
 - Test indexing and query performance
- *API Testing:*
 - Validate successful implementation of REST API endpoints (GET, POST, CREATE, DELETE)
 - Test API response times
 - Verify error handling for invalid requests and data
 - Ensure proper authentication and security tests (CORS)

Front-End Testing:

- *UI/UX Testing:*
 - Test navigation flow and user interaction consistency
 - Validate form inputs, click events, dropdown selections, and data entry restrictions
 - Validate useability of the GUI
- *Functionality Testing:*
 - Test real-time updates for stock levels and notifications
 - Validate actions delegated in the GUI perform requests delegated to the database through the API
 - Confirm smooth handling of bulk action (batch update, bulk deletions)

Classes & Algorithms:

- *Functionality Testing:*
 - Develop unit tests for individual class function and validate after creation of each function
 - Test validation logic for product categories, stock levels, and expiration dates
 - Check algorithms for product aging alerts and automatic reorder triggers

Project Management

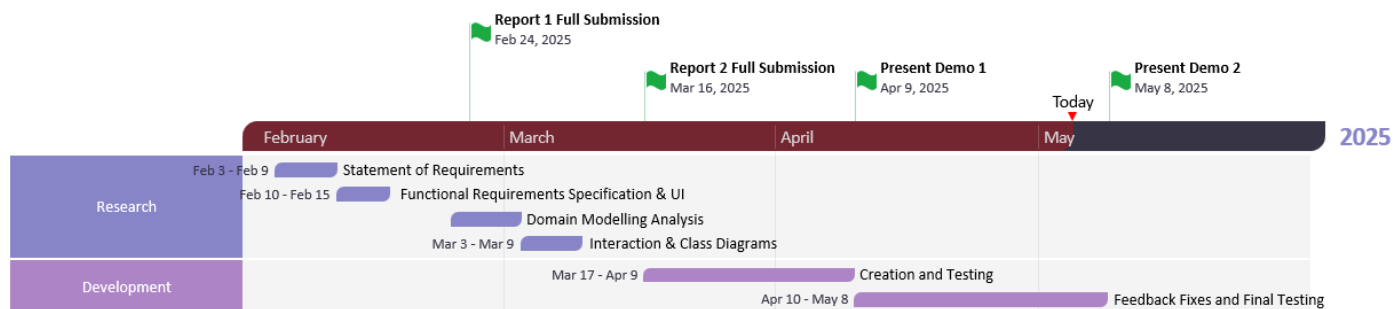
Merging the Contributions from Individual Members

Throughout the project, the team has effectively integrated each component by aligning our ideas with one another, ensuring a shared understanding of the overall vision and goals. This collaboration has been particularly engaging, as it requires reviewing and building upon each other's work rather than working in isolation. The process has fostered a deeper integration of perspectives, allowing us to blend the ideas of four individuals into a cohesive system.

History of Work, Current Status, and Future Work

History of Work

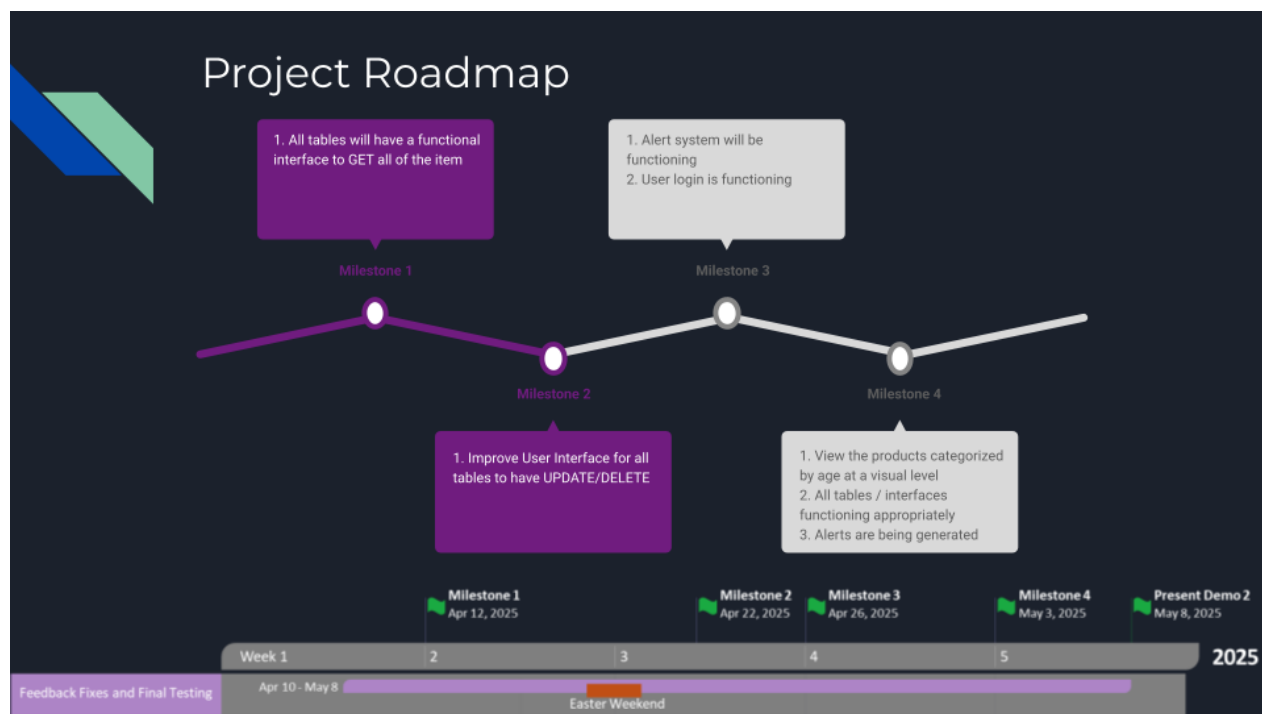
We collaborate with dedicated weekly meetings. During these meetings, we review the weekly deliverables outlined in the WasteLess Gantt. You can see a preview in the below screenshot or review the entire gantt with this link [WasteLesss Gantt.xlsx](#). We divide the deliverables evenly among team members, and update the Task Assignment Tracker found at this link [Task Assignment Tracker.xlsx](#) with outstanding tasks.



WASTLESSS GANTT CHART																																															
TASK NAME	START DATE	END DATE	DURATION (WORK DAYS)	TEAM MEMBER	PERCENT COMPLETE	WEEK 7			WEEK 8					WEEK 9					WEEK 10					WEEK 11					WEEK 12																		
						Th	F	S	T	W	Th	F	S	T	W	Th	F	S	T	W	Th	F	S	T	W	Th	F	S	T	W	Th	F	S														
Project Management	3/11	3/15	4	Larisa	100%																																										
Merging the Contributions from Individual Team	3/11	3/15	4	Team	100%																																										
Project Coordination	3/11	3/15	4	Team	100%																																										
Progress Report	3/11	3/15	4	Team	100%																																										
Plan of Work	3/11	3/15	4	Larisa	100%																																										
Breakdown of Responsibilities	3/11	3/15	4	Larisa	100%																																										
Proof-reading and Editing	3/15	3/16	1	Team	100%																																										
Demo 1																																															
Task Overview and Delegation	3/18	3/18	1	Team	100%																																										
Front End - CSS/HTML	3/18	4/8	16	Brandon	100%																																										
Backend - API Node.js / Testing	3/18	4/8	16	Larisa/Pavel	100%																																										
Database Creation / Testing	3/18	4/8	16	Janelle	100%																																										
Integration Testing	3/18	4/8	16	TBD	100%																																										
Alerts	3/18	4/8	16	TBD	100%																																										
User Account / Login	3/18	4/8	16	TBD	100%																																										
Proof-reading and Editing	4/8	4/9	1	Team	100%																																										
Demo 2																																															
Task Overview and Delegation	4/12	4/12	1	Team	100%																																										
Front End - CSS / HTML - update and finish tables	4/12	5/7	18	Brandon	100%																																										
Alerts and Alert Logic	4/12	5/7	18	Janelle	100%																																										
Email Notification	4/12	5/7	18	Larisa	100%																																										
User Login	4/12	5/7	18	Brandon	100%																																										
Report 3 - Design Patterns	4/12	5/4	15	Larisa	100%																																										
Report 3 - OCL Contracts	4/12	5/4	15	Pavel	100%																																										
Report 3 Final Edits - Submission	4/12	5/4	15	Team	100%																																										
Final System Testing	4/12	5/7	18	Team	100%																																										
Present Demo 2	5/7	5/8	1	Team	25%																																										

The project started with a broad overview for the project deliverables, but through continuous discussion and collaborative teamwork we developed more individualized goals. These were defined and tracked in our Module Class tracker shown below in the “Coding Responsibilities” section. The full tracker can be found at [Module Class Tracker.xlsx](#).

After Demo 1 a more granular timeline was defined to ensure all of the open deliverables were met prior to final Demo 2 submission which can be seen below.



Coding Responsibilities

	Owner					Owner				Frontend Development				Owner				Database Management			
Responsibility	Module		Functions	Commit Status (Github)	Testing Status (Postman)	Responsibility	Module		Language	Class	Testing Status	Responsibility	Module		Language	Class	Queries	Response			
Larisa	vendorController	GET / POST	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	userAccount / Login	All functions tested	Janelle	Product	SQL	Inventory, AgingInventory	Created table, will add item profile						
Larisa	vendorRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Vendor	All functions tested	Janelle	Order	SQL	Orders, OrderItems	Created both tables with relationships						
Pavel	companyDivisionController	GET / POST	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Inventory	All functions tested	Janelle	SearchTool	SQL	No table needed (handled in backend/frontend)	No table needed						
Pavel	companyDivisionRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Company Division	All functions tested	Janelle	Alert	SQL	Alerts	Created alert table						
Larisa	locationController	GET / POST	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Location	All functions tested	Janelle	Vendor	SQL	Vendor, Shipping	Created tables for supplies and						
Larisa	locationRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Aging Inventory	All functions tested	Janelle	Customer	SQL	Customer	Created customer table						
Larisa	inventoryController	GET / POST	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Customer	All functions tested	Janelle	UserAuth	SQL	UserAccount	Created Table						
Larisa	inventoryRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning		Brandon	User Interface	JavaScript/CSS/HTML	Order	All functions tested	Janelle	Location Management	SQL	Location	Created Table						
Larisa	agingInventoryController	GET / POST	Committed	All tested and functioning		Janelle	User Interface	JavaScript/CSS/HTML	OrderItems	All functions tested	Janelle	Company Structure	SQL	Company/Division	Created Table						
Larisa	agingInventoryRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning		Janelle	User Interface	JavaScript/CSS/HTML	Alerts	All functions tested											
Pavel	customerController	GET / POST	Committed	All tested and functioning		Larisa	User Interface	JavaScript/CSS/HTML	Shipping	All functions tested											
Pavel	customerRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Pavel	orderController	GET / POST	Committed	All tested and functioning																	
Pavel	orderRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Larisa	orderItemsController	GET / POST	Committed	All tested and functioning																	
Larisa	orderItemsRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Larisa / Janelle	alertController	GET / POST	Committed	All tested and functioning																	
Larisa / Janelle	alerRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Pavel	userAccountController	GET / POST	Committed	All tested and functioning																	
Pavel	userAccountRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Larisa	ShippingController	GET / POST	Committed	All tested and functioning																	
Larisa	shippingRoutes	GET/PUT/POST/DELETE	Committed	All tested and functioning																	
Larisa	emailController	GET	Committed	All tested and functioning																	
Larisa	alertEmailJob	GET	Committed	All tested and functioning																	

Future Work

As the system continues to evolve, there are numerous opportunities to enhance its functionality, usability, and integration within broader business operations. While the initial scope laid a strong foundation, some of the more ambitious initial goals were unable to fully be achieved. Future development can significantly increase the system's value by addressing emerging business needs, streamlining workflows, and leveraging modern technologies. Below are several potential areas of improvement and expansion that could further strengthen the system and support long-term scalability and efficiency.

Report Enhancements

- **Technical Documentation Updates:** The *Data Types and Operation Signatures* section may need to be revised to reflect our current technology stack. The current documentation uses Java-style types (e.g., int, string), while the implementation is based on JavaScript and PostgreSQL. This should be reviewed with the team to ensure consistency and accuracy for future development.

Integration & Interoperability

- **ERP & CRM Integration:** Link with common business systems like SAP, Oracle, or Salesforce to centralize data and automate procurement, invoicing, and customer relationship management.
- **Supplier Portal Access:** Enable suppliers to update inventory or pricing directly through a secure interface including broadcasting product sales or lead times for new product launches.

Data & Intelligence

- **Stock Availability Checks:** A key planned feature is validating stock availability before processing orders. Although referenced in the *Mathematical Model* section, this is not yet implemented and will be a critical addition to improve order accuracy and inventory management.
- **Advanced Analytics Dashboard:** Although our system provided a visual system for viewing products, it could be improved by adding interactive dashboards with graphs and charts comparing supplier performance, sales trends, and inventory turnover rates.
- **More Cost Optimization & Forecasting Recommendations:** Additional machine learning integration to predict product demand, while also offering the most cost-effective supplier based on price, delivery time, and historical reliability.

User Experience & Feedback

- **Email Add Useability:** Currently the system sends emails every 24hours, but there is not an easy way to add the email address on the user interface, the email address has to be added from the back-end. An easier way to update the email address on the user interface would make the system more user friendly.
- **In-app Guidance & Tips:** Provide on-site directives or add learning modules to help users navigate complex features.
- **Feedback & Suggestions:** Implement a voting or survey mechanism to allow for users to upvote other suggestions or improvements for the system.

References

- Dyson, J. (2019, January 7). Conjoining FURPS and MOSCOW to analyse and prioritise requirements. <https://www.linkedin.com/pulse/conjoining-furps-moscow-analyse-prioritise-jonathan-dyson>
- Erlenkotter, D. (1990). Ford Whitman Harris and the Economic Order Quantity Model. *Operations Research*, 38(6), 937–946. <https://doi.org/10.1287/opre.38.6.937>
- Fernando, J. (2024b, June 12). Economic Order Quantity: What Does It Mean and Who Is It Important for? Investopedia. <https://www.investopedia.com/terms/e/economicorderquantity.asp>
- GeeksforGeeks. (2024, December 28). Multithreading in operating system. GeeksforGeeks. <https://www.geeksforgeeks.org/multithreading-in-operating-system/>
- Ian McCue (2020, November 5) Inventory Carrying Costs: What It Is & How to Calculate It <https://www.netsuite.com/portal/resource/articles/inventory-management/inventory-carrying-costs.shtml#:~:text=Typical%20holding%20costs%2C%20another%20name,an%20item%20before%20selling%20it.>
- Jenkins, A. (2024, December 6). ABC Analysis in Inventory Management: Benefits & best practices. Oracle NetSuite. <https://www.netsuite.com/portal/resource/articles/inventory-management/abc-inventory-analysis.shtml#:~:text=ABC%20analysis%20is%20an%20inventory,classes%20based%20on%20those%20criteria.>
- Šimek, V. (2022, May 16). Inventory aging: Recognizing and preventing it | Inventoro. Inventoro. <https://inventoro.com/inventory-aging-recognizing-and-preventing-it/>